

第 4 章 树

郑冠杰

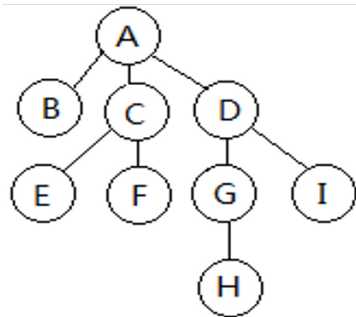
2026 年 3 月 23 日

- ▶ 树
- ▶ 二叉树
- ▶ 遍历序列确定二叉树
- ▶ 最优二叉树
- ▶ 树和森林
- ▶ 二叉线索树 *
- ▶ 表达式树 *
- ▶ 等价关系 *
- ▶ 优先级队列

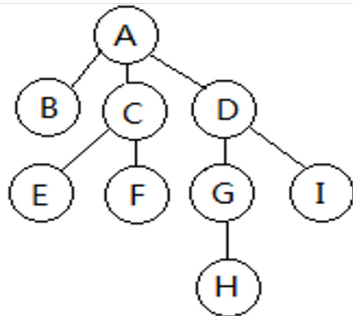
树（两种定义方式）

- ▶ 在一个元素集合中，元素呈上下层次关系。对一个结点而言，上层元素为其直接前驱且直接前驱唯一，下层元素为其直接后继且直接后继可以有多个，这样的结构就是**树结构**。
- ▶ 树是有限个 ($n > 0$) 元素组成的集合。在这个集合中，有一个结点称为根；如果有其他的结点，这些结点又被分为若干个互不相交的非空子集，每个子集又是一棵树，称为根的子树；每个子树都有自己的根，子树的根称为树根结点的孩子结点。

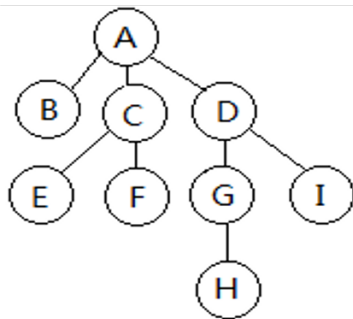
- 一个结点的子树的根称为该结点的**孩子结点**或**儿子结点**；反之，相对于孩子结点，该结点称为**父结点**。父结点的父结点称为**祖父结点**，从根到树中某个结点的路径上经过的所有结点，包括根结点，都称为这个结点的**祖先结点**；相对地，这些结点称为祖先结点的**子孙结点**。同一父结点的结点互为**兄弟结点**，同一祖父但不同父亲的结点互称为**堂兄弟结点**。



- 树中每个结点拥有的孩子结点的个数称为该结点的**度**，度为 0 的结点称**叶子结点**或**终端结点**，度不为 0 的结点称**非叶子结点**或**中间结点**或**非终端结点**。**树的度**是树中每个结点的度的最大值。
- 树中结点具有层次关系。**根**的层次数通常规定为 1，其余结点的层次数是其父结点的层次数加 1。树中所有结点的层次数的最大值就是**树的高度**（注意：在有些教科书上，树高定义为结点的最大层次数减一（即分支的层次数））。



- 对树中的任意一个结点，如果其孩子结点都被规定了一定的顺序，如谁是第一个孩子、谁是第二个孩子等，这棵树就称**有序树**。如果结点的孩子没有规定顺序，称为**无序树**。
- 两棵及以上的树称为**森林 (Forest)**。
- 在树中，父结点可以看作是孩子结点的直接前驱、孩子结点可以看作是父结点的直接后继，直接前驱是唯一的、直接后继可以有多个。



树的抽象数据类型

数据及关系:

有限 ($n > 0$) 个相同类型的元素组成的集合。其中一个元素称为根; 如果还有其余元素, 则这些元素被分为若干个互不相交的非空子集, 每个子集是一棵子树, 每个子树有自己的根, 子树的根是树根的孩子。

操作:

Constructor: 前提: 已知结点的数据元素值和结点间树形关系。结果: 创建一棵树。

GetRoot: 前提: 已知一棵树。结果: 得到树的根结点。

树的抽象数据类型

FirstChild:

前提：已知树中的某一结点 p 。结果：得到结点 p 的第一个儿子结点。

NextChild:

前提：已知树中的某一结点 p 和它的一个儿子结点 u 。

结果：得到结点 p 的、儿子结点 u 右侧的第一个儿子结点 v 。

Search: 前提：已知关键字 key 。结果：返回具有关键字 key 的结点。

InsertChild:

前提：已知某结点 p 及新结点的数据值 $value$ 。

结果：据 $value$ 值创建一个新结点 q , 将其作为 p 的下一个儿子插入。

树的抽象数据类型

DeleteChild:

前提：已知某结点 p 及它的儿子结点的序号 k 。

结果：删除结点 p 的第 k 个儿子结点。

Traverse

前提：一棵树

结果：访问树中每一结点，且每个结点只访问一次。

树的物理结构

顺序结构？顺序结构中元素值的存储容易，元素间的层次关系如何用顺序结构来存储？一个想法是在存储元素的分量中附加表明其父子关系的信息，即每个结点除了存储元素的值还存储父子关系的信息。由于树中每个结点的孩子个数都不一样，结点结构中设置几个字段来保存孩子结点的地址信息？预留多了浪费空间，预留少了无法增加孩子结点，用顺序结构不经济。

链式结构？因每个结点的地址不连续，更要通过设置附加的字段来指明父子关系，显然它 also 存在着结点中孩子指针个数不好预估的问题。

没有合适的物理存储做基础，树的基本操作就无从谈起。

- ▶ 树
- ▶ 二叉树
- ▶ 遍历序列确定二叉树
- ▶ 最优二叉树
- ▶ 树和森林
- ▶ 二叉线索树 *
- ▶ 表达式树 *
- ▶ 等价关系 *
- ▶ 优先级队列

二叉树

- 二叉树的定义
- 二叉树的性质
- 二叉树的存储
- 二叉树类及操作实现
- 二叉树的遍历

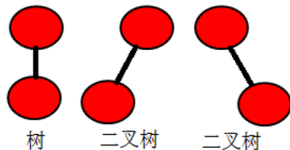
二叉树的定义

二叉树 (Binary Tree) 是结点的有限集合。它或者为空, 或者由一个根结点及两棵互不相交的左、右子树构成, 其左、右子树又都是二叉树。

- 注意 1: 二叉树必须严格区分左右子树。

即使只有一棵子树, 也要说明它是左子树还是右子树。交换一棵二叉树的左右子树后得到的是另一棵二叉树。

- 注意 2: 二叉树定义与树的定义的差别。



二叉树定义与树的定义的差别

- 二叉树中结点个数可以为 0，允许一棵空二叉树存在，这是作为工具能更方便地使用；而树中结点个数不能为 0，必须至少是 1。
- 二叉树中左右孩子要明确指出是左还是右，即便只有一个孩子，也要指明它是左子还是右子；有序树中孩子只是进行了排序，没有左右之分，当某个结点只有一个孩子时，只能说明它是大孩子，不需要确定其是左是右。

二叉树的基本形态



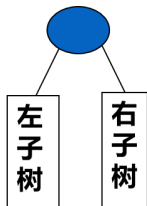
(a)

空二叉树



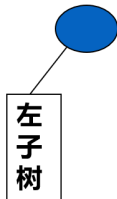
(b)

根和空左右子树



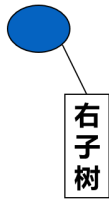
(c)

根和左右子树



(d)

根和左子树



(e)

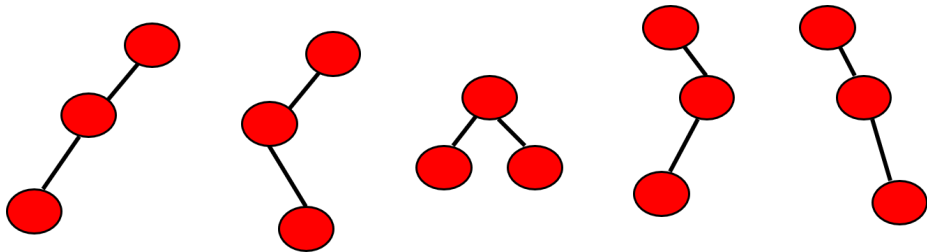
根和右子树

二叉树的基本形态

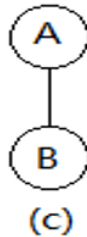
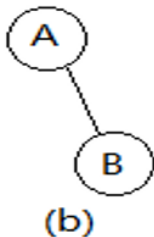
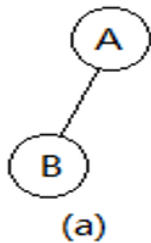
- 结点总数为3的所有二叉树的不同形状；
- 请大家在纸上画出所有的形状。

二叉树的基本形态

- 结点总数为3的所有二叉树的不同形状：

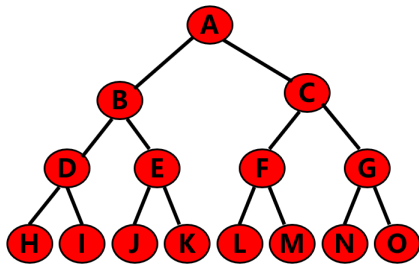


二个结点的二叉树和树：

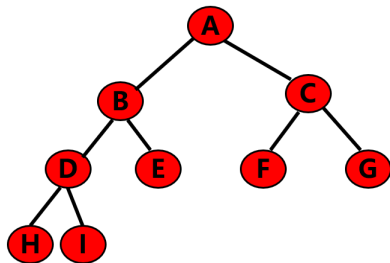


满二叉树

- 一棵二叉树中任意一层的结点个数都达到了最大值，即：一棵高度为 k 并具有 $2^k - 1$ 个结点的二叉树。



满二叉树实例



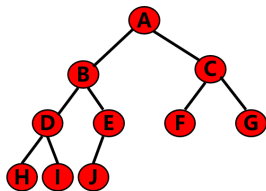
不是满二叉树

完全二叉树

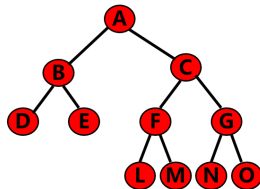
在满二叉树的最底层自右至左依次去掉若干个结点就是**完全二叉树**。

- 完全二叉树的特点：

- 1) 所有的叶结点都出现在最低的两层上；
- 2) 对任一结点，如果其右子树的高度为 k ，则其左子树的高度为 k 或 $k+1$ 。



完全二叉树



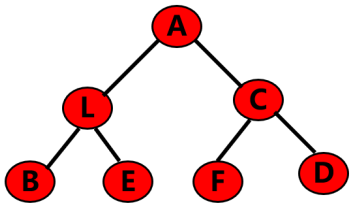
非完全二叉树

二叉树

- 二叉树的定义
- 二叉树的性质
- 二叉树的存储
- 二叉树类及操作实现
- 二叉树的遍历

二叉树的性质 1

一棵非空二叉树的第 i 层上最多有 2^{i-1} 个结点 (层数: 从根向下数, $i \geq 1$)。



数学归纳法证明

- 1 层: 结点个数 $2^{1-1} = 1$ 个
- 2 层: 结点个数 $2^{2-1} = 2$ 个
- 3 层: 结点个数 $2^{3-1} = 4$ 个

二叉树的性质 1

证明:

当 $i = 1$ 时, 只有一个根结点, $2^{i-1} = 2^0 = 1$, 命题成立。

假定对于所有的 j , $1 \leq j \leq k$, 命题成立。即第 j 层上至多有 2^{j-1} 个结点。

由归纳假设可知, 第 $j = k$ 层上至多有 2^{k-1} 个结点。

若要使得第 $k+1$ 层上结点数为最多, 那么, 第 k 层上的每个结点都必须有两个儿子结点。

因此, 第 $k+1$ 层上结点数最多为第 k 层上最多结点数的二倍, 即: $2 \times 2^{k-1} = 2^{k+1-1} = 2^k$ 。

所以, 命题成立。

二叉树的性质 2

一棵高度为 k 的二叉树，最多具有 $2^k - 1$ 个结点。

证明：

这棵二叉树中的每一层的结点个数必须最多。

根据性质 1，第 i 层的结点数最多等于 2^{i-1} 。

因此结点个数 N 最多为：

$$\sum_{i=1}^k 2^{i-1} = 2^k - 1$$

二叉树的性质 3

对于一棵非空二叉树，如果叶子结点数为 n_0 ，度数为 2 的结点数为 n_2 ，则有 $n_0 = n_2 + 1$ 成立。

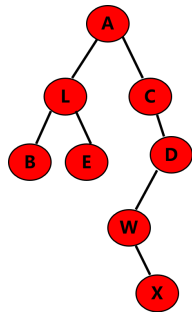
证明：

令 n_1 为度数为 1 的结点数，**总结点** $n = n_0 + n_1 + n_2$ ，

从下往上看，二叉树中除了根结点，每个结点向上都通过唯一的分支和父结点相连，所以二叉树中共有 $n-1$ 条分支。（看前驱）

从上往下看，每个结点都通过向下的分支和孩子结点相连，度为 1 的结点向下发出 1 个分支，度为 2 的结点向下发出 2 个分支，故： $n-1 = 1 * n_1 + 2 * n_2$ 。（看后继）

从而有 $n_0 = n_2 + 1$ 。



二叉树的性质 4

具有 n 个结点的完全二叉树的高度 $k = \lfloor \log_2 n \rfloor + 1$ 。

证明：

根据完全二叉树的定义和性质 2 可知，

高度为 k 的完全二叉树的第一层到第 $k-1$ 层具有最多
结点个数的总数为 $2^{k-1} - 1$ 个，

第 k 层至少具有一个结点，至多有 2^{k-1} 个结点。因此：

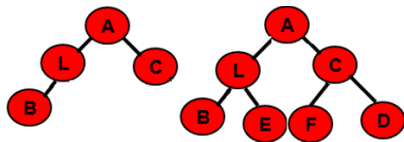
$$2^{k-1} \leq n < 2^k$$

对不等式取对数，有：

$$k-1 \leq \log_2 n < k$$

由于 k 是整数，所以有：

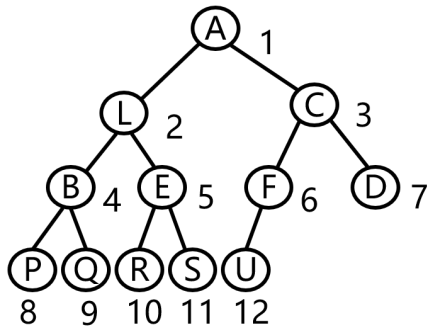
$$k = \lfloor \log_2 n \rfloor + 1$$



二叉树的性质 5

如果对一棵有 n 个结点的完全二叉树中的结点按层**自上而下**（从第 1 层到第 $\lfloor \log_2 n \rfloor + 1$ 层），每一层按**自左至右**依次编号，若设根结点的编号为 1，则对任一编号为 i 的结点 ($1 \leq i \leq n$)，有：

- ① 如果 $i = 1$ ，则该结点是二叉树的根结点；如果 $i > 1$ ，则其父亲结点的编号为 $\lfloor i/2 \rfloor$ ；
- ② 如果 $2i > n$ ，则编号为 i 的结点为叶子结点，没有儿子；否则，其左儿子的编号为 $2i$ ；
- ③ 如果 $2i + 1 > n$ ，则编号为 i 的结点无右儿子；否则，其右儿子的编号为 $2i + 1$ 。



二叉树的性质 5

证明：利用数学归纳法证明。

当 $i=1$ ，即为根结点时，根的左子编号为 2、右子编号为 3，结论显然成立。

设 $i=k$ 时，其左子存在且编号为 $2k$ 、右子存在且编号为 $2k+1$ ，则编号为 $k+1$ 的结点因 $k+1 \leq 2k$ 就一定存在。如果编号为 $k+1$ 的结点有左子，左子一定紧挨着编号为 k 的结点的右子，下标即为 $2k+1+1=2(k+1)$ ；如果编号为 $k+1$ 的结点又有右子则其编号为其左子编号加一，为 $2(k+1)+1$ 。在完全二叉树中， n 个结点是从 1 开始连续编号的，结点的编号最大为 n ，当某个结点存在，其编号必 $\leq n$ ，如果计算出某结点编号大于 n ，就说明该结点不存在。

结合以上讨论，根据数学归纳法，性质 5(2)、(3) 成立。

二叉树的性质 5

根据结论 (2), 如果一个结点是某个结点 j 的左子, 则其编号有 $2j$ 和 j 的关系;
根据结论 (3), 如果一个结点是某个结点 j 的右子, 则其编号有 $2j+1$ 和 j 的关系, 故如果一个非根结点的编号是 i , 其父结点的编号就是 $\lfloor i/2 \rfloor$, 性质 5(1) 得证。

二叉树性质

1. 二叉树第 i 层结点数：最多 2^{i-1}
2. 二叉树总结点数： $2^k - 1$
3. 二叉树度数与结点关系： $n_0 = n_2 + 1$
4. 完全二叉树高度： $k = \lfloor \log_2 n \rfloor + 1$
5. 完全二叉树结点编号与结点的关系

经典例题

- 记高度为 h 的二叉树（只含有一个结点的二叉树高度定义为 1）中可能包含的最小结点数为 m ，最大结点数为 M ，则 (m, M) 是？
A. $(\log h, 2^h)$ B. $(\log h, 2^h - 1)$ C. $(h, 2^h)$ D. $(h, 2^h - 1)$
- 对任意一棵由 n 个结点构成的树，这 n 个结点的度数之和是多少？
A. $n - 1$ B. $2n - 1$ C. n D. 不确定

经典例题

- 记高度为 h 的二叉树（只含有一个结点的二叉树高度定义为 1）中可能包含的最小结点数为 m ，最大结点数为 M ，则 (m, M) 是？ D
A. $(\log h, 2^h)$ B. $(\log h, 2^h - 1)$ C. $(h, 2^h)$ D. $(h, 2^h - 1)$
- 对任意一棵由 n 个结点构成的树，这 n 个结点的度数之和是多少？ A
A. $n - 1$ B. $2n - 1$ C. n D. 不确定

二叉树的基本操作

- 构造类：在内存中创建一棵二叉树
- 属性类：判二叉树空、获取根结点、求以某结点为根的二叉树中的结点个数和高度。
- 数据操纵类：对二叉树的整体删除。对于二叉树结点的插入、删除操作，因不知其具体意义和要求，将它放到有实际插入、删除意义的后续章节中讲解。
- 遍历类：按前序、后序、中序、层次遍历二叉树的操作。遍历类操作非常重要，在下一节中详细讨论。

二叉树

- 二叉树的定义
- 二叉树的性质
- 二叉树的存储
- 二叉树类及操作实现
- 二叉树的遍历

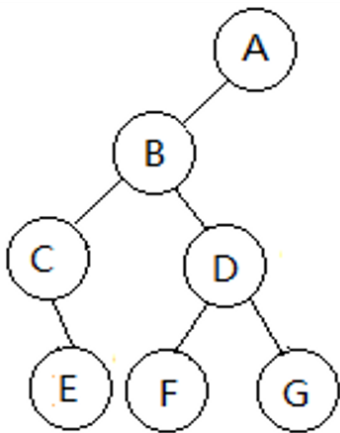
二叉树的物理存储实现

- 顺序结构
- 链式结构

二叉树的顺序存储

- ▶ 用一组连续的空间即数组来存储二叉树中的结点。
- ▶ 每个结点除了包括元素值，还包括表达二叉树父子关系的字段。可设置四个字段：data、left、right、parent，其中 left、right、parent 为其左、右孩子及父结点在数组中的下标。
- ▶ 当一个结点没有孩子结点时，结点的 left、right 字段设置为-1；当一个结点没有父结点时，结点的 parent 字段设置为-1。
- ▶ 结点在存储时，可以随意地按照任何顺序将它们存储在数组中，元素之间的关系全靠字段 left、right 和 parent 来维系。

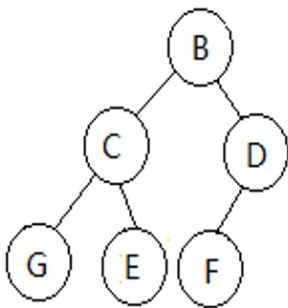
二叉树的顺序存储



	data	left	right	parent
0	A	1	-1	-1
1	B	2	3	0
2	C	-1	4	1
3	D	5	6	1
4	E	-1	-1	2
5	F	-1	-1	3
6	G	-1	-1	3

二叉树的顺序存储

一棵完全二叉树，可以更简单。具体方法是：先对结点按照二叉树层次自上而下、自左向右进行编号，编号从 0 开始逐步加一，然后将结点存储在下标和其编号相同的数组分量中。



(a)

	data	left	right
0	B	1	2
1	C	3	4
2	D	5	-1
3	G	-1	-1
4	E	-1	-1
5	F	-1	-1

(b)

	data
0	B
1	C
2	D
3	G
4	E
5	F

(c)

二叉树的顺序存储

顺序存储方式：

好处是数组访问简单，
坏处是它需要事先预估出数据的最大规模。

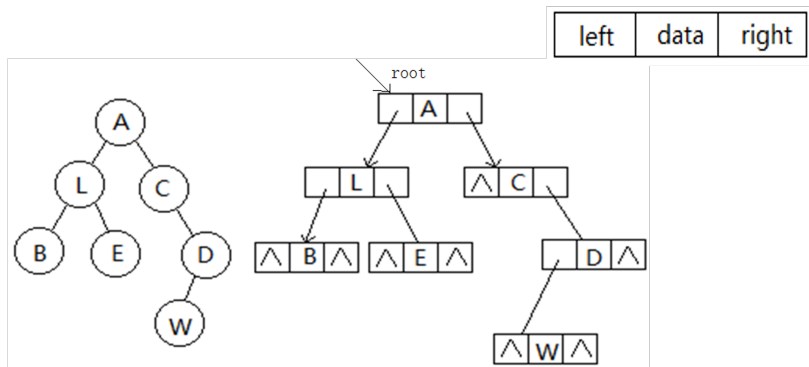
二叉树的物理存储实现

- 顺序结构
- 链式结构

二叉树的链式存储

两种形式：标准形式，广义标准形式。

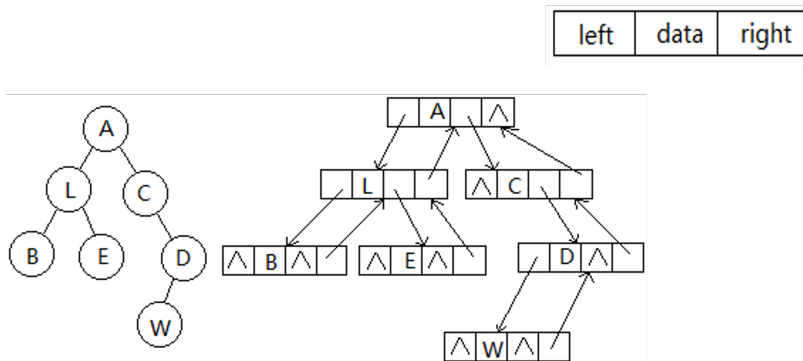
► 标准形式



二叉树的链式存储

两种形式：标准形式，广义标准形式。

► 广义标准形式



二叉树的链式存储

因为求结点的父结点的概率并不大，在标准形式中也依然能求得一个结点的父结点，因此通常多用标准形式。

如求值为 x 的结点的父结点，可以通过遍历逐个检查哪个结点的孩子结点为 x ，这个结点就是 x 的父结点。

标准形式的链式存储形式最为直观。

二叉树

- 二叉树的定义
- 二叉树的性质
- 二叉树的存储
- 二叉树类及操作实现
- 二叉树的遍历

标准存储的二叉树类中属性的定义

BTree

```
#ifndef BTREE_H_INCLUDED
#define BTREE_H_INCLUDED
#include <iostream>
#include "seqStack.h"
#include "seqQueue.h"
using namespace std;
//BTree类的前向说明
template <class elemType>
class BTree;
template <class elemType>
class Node
{
    friend class BTree<elemType>;
private:
    elemType data;          Node *left, *right;
public:
    Node(){left=NULL; right=NULL;};
    Node(const elemType &e, Node* L=NULL, Node *R=NULL)
    { data = e; left=L; right=R; };
};
```

标准存储的二叉树类中属性的定义

BTree

```
template <class elemType>
class BTree
{
    private:
        Node<elemType> *root;
        int Size (Node<elemType> *t); //求以t为根的二叉树的结点个数。
        int Height (Node<elemType> *t); //求以t为根的二叉树的高度。
        void DelTree(Node<elemType> *t); //删除以t为根的二叉树
        void PreOrder(Node<elemType> *t);
        // 按前序遍历输出以t为根的二叉树的结点的数据值
        void InOrder(Node<elemType> *t);
        // 按中序遍历输出以t为根的二叉树的结点的数据值
        void PostOrder(Node<elemType> *t);
        // 按后序遍历输出以t为根的二叉树的结点的数据值
}
```

标准存储的二叉树类中属性的定义

BTree

```
public:
    BTree(){root=NULL;}
    void createTree(const elemType &stopFlag); //创建一棵二叉树
    bool isEmpty () { return (root == NULL); } // 二叉树为空返回 true, 否则返回 false
    Node<elemType> * GetRoot(){ return root; }
    int Size (); //求二叉树的结点个数。
    int Height (); //求二叉树的高度。
    void DelTree(); //删除二叉树
    void PreOrder(); // 按前序遍历输出二叉树的结点的数据值
    void InOrder(); // 按中序遍历输出二叉树的结点的数据值
    void PostOrder(); // 按后序遍历输出二叉树的结点的数据值
    void LevelOrder(); // 按中序遍历输出二叉树的结点的数据值
};
```

两种情况：

- ① 构造一棵空的二叉树：属性 `root = NULL`;
- ② 建立一棵非空二叉树

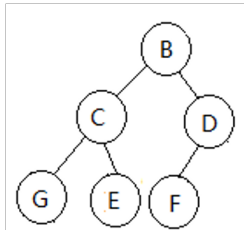
本章不讨论插入、删除，空二叉树是无法验证其各种属性类操作，故要建立非空二叉树。

建立一棵非空二叉树

只有在内存中建立起一棵二叉树，其余的操作才便于测试。

具体目标：

- ① 在内存中为每个元素创建存储结点空间，元素值放该结点中。
- ② 建立结点的父子关系，即填写左右子结点地址。



龙门阵：

主动建根结点，由根（父）引出左右子结点的创建，左右子排队下延。此处可对照单枝树思考。

建立一棵非空二叉树

算法思路：

借助一个队列来管理结点。

读入根结点的值，在内存中创建根结点并将根结点地址进队。

通过将结点从队列中逐个出队、按照出队结点的信息提醒用户输入其孩子结点信息、为孩子创建结点空间，并将孩子结点的地址写到父结点的左右孩子字段里，然后将孩子结点地址进队，让它们在队中等待出队的机会，以便有机会创建它们自己的孩子结点。如此反复进行以上出队、输入孩子、建立孩子结点、链接孩子结点、孩子结点进队的操作。当整个队列为空时，循环结束，在内存中就建好了一棵二叉树对应的二叉链表。

建立二叉树算法分析

- ▶ 二叉树的第一层结点（根）是主动进队的。
- ▶ 根出队时，输入、创建并链接了根的孩子（第二层的所有结点），并使得第二层结点全部进队；
- ▶ 当第二层结点逐个出队时，又输入、创建并链接了第三层的所有结点；
- ▶ 如此这般，所有层的结点得以创建并链接在父结点的左或者右孩子链上。

创建一棵树

- 创建过程：

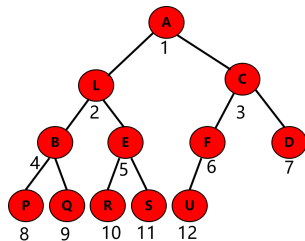
- ◇ 先输入根结点的值，创建根结点；
- ◇ 对已添加到树上的每个结点，依次输入它的两个儿子的值。如果没有儿子，则输入一个特定标志（stopFlag）表示终止。

- 实现工具：

- ◇ 使用一个队列，将新加入到树中的结点放入队列；
- ◇ 依次出队。对每个出队的元素输入它的儿子，入队。

创建一棵树

1. 建立队列，队列中元素为**结点的地址**；
2. 主动输入**根结点值**，建结点，**进队**；
3. 逐步**出队**，输入孩子结点，**孩子结点地址进队**；
4. 反复进行 3，直到队空。



A							
L	C						
C	B	E					
B	E	F	D				
E	F	D	P	Q			
F	D	P	Q	R	S		
D	P	Q	R	S	U		
P	Q	R	S	U			
Q	R	S	U				
R	S	U					
S	U						
U							

队列的变化

建立一棵非空二叉树实现程序

CreateTree

```
template <class elemType>
void BTree<elemType>::createTree(
    const elemType &stopFlag)
//创建一棵二叉树
{   seqQueue<Node<elemType>*> que;
    elemType e, el, er;      Node<
        elemType> *p, *pl, *pr;

    cout<<"Please input the value of
        the root: ";
    cin>>e;
    if (e== stopFlag) {return;}
    p = new Node<elemType>(e);
    root = p; //根结点为该新创建结点
    que.enqueue(p);
```

```
    while (!que.isEmpty())
    {   p = que.front(); //获得队首元素并出队
        que.dequeue();
        cout<<"Please input the value of the left
            child and the right child of "<<p->data
            <<" using "<< stopFlag <<" as no child: ";
        cin>>el>>er;
        if (el!= stopFlag) //该结点有左孩子
        {   pl = new Node<elemType>(el);
            p->left = pl;      que.enqueue(pl);
        }
        if (er!= stopFlag) //该结点有右孩子
        {   pr = new Node<elemType>(er);
            p->right = pr;    que.enqueue(pr);
        }
    }
}
```

求属性的两个操作：函数 Size 和 Height。

Size 操作：如果二叉树为空，返回 0，否则返回根的个数 1 加上根的左、右子树中结点个数；

Height 操作：如果二叉树为空，返回 0，否则返回根的高度 1 加上根的左、右子树高度中的最大值。

从求 Size、Height 算法可以感受二叉树操作中递归应用的普遍性

原因：二叉树的递归定义。

一般来说，只要能用递归来定义该操作，就能写出递归的算法。

递归算法逻辑简单明了、代码较短，不容易出错。

递归是计算思维最典型的组成元素之一。

递归程序要素

- ① 简单问题规模，不需要递归，直接可解决。
- ② 规模大的问题在规模小的问题解的基础上求解。
- ③ 函数调用一步步由规模大的问题向规模小的问题上发展。

常见错误：在 2 上，公式或者返回方式错误。后面例子中详解。

递归程序简单例子

屏幕上输出 n 行 ***

```
void print(int n)
{   if (n<=0) return;
    cout<< "***" <<endl;
    print(n-1) ;
}
```

求 n!

```
int p(int n)
{   if (n<=1) return 1;
    return n*p(n-1);
}
```

公有和私有函数重载问题：如，Size 操作

- 递归算法中参数需要体现出规模的变化，这里用一个结点地址，表示以该结点为根的子树。
- 从类的属性中可以看出，`root` 为私有成员，故带参数的 `size` 的递归函数并不方便外部函数调用，故该递归函数设置为私有。
- 不带参数的 `size` 函数，虽不利于递归，但有利于外部函数调用，故设置为公有。
- 公有成员函数 `size` 可以通过调用私有成员函数 `size` 得以实现。
- `size` 并非一定要用递归算法，可借助后面非递归遍历算法方便地实现。

属性类

Size

```
template <class elemType>    //公有
int BTree<elemType>::Size()
{ return Size(root); }

template <class elemType>    //私有
int BTree<elemType>::Size (Node<
    elemType> *t)
//得到以t为根二叉树结点个数，递归算法实现。
{ if (!t) return 0;
  return 1+Size(t->left)+Size(t->right); }
```

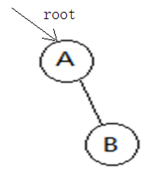
时间效率分析：

每次执行两个 return 二选一；每个结点经历一次 1+。。。。

每个空链域经历一次 0；共 $n+(2n-(n-1)) = 2n+1$ 次

出错情况：

1. $\text{size}(t \rightarrow \text{left}); \text{size}(t \rightarrow \text{right});$
2. $1 + \text{size}(t \rightarrow \text{left}) + \text{size}(t \rightarrow \text{right})$

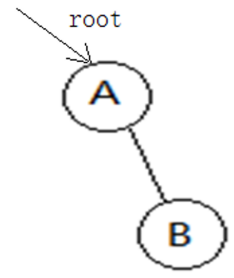


属性类

Height

```
template <class elemType>
int BTree<elemType>::Height()
{ return Height(root); }

template <class elemType>
int BTree<elemType>::Height(Node<elemType> *t)
//得到以t为根二叉树的高度，递归算法实现。
{ int hl, hr;    if (!t) return 0;
  hl = Height(t->left);    hr = Height(t->right);
  if (hl>=hr) return 1+hl;    return 1+hr; }
```



二叉树类的定义

BTree

```
template <class elemType>
void BTree<elemType>::DelTree()
{ DelTree(root);    root = NULL;    }

template <class elemType>
void BTree<elemType>::DelTree(Node<elemType> *t)
//删除以t为根的二叉树，递归算法实现
{ if (!t) return;    DelTree(t->left);    DelTree(t->right);
  delete t;    }
```

二叉树

- 二叉树的定义
- 二叉树的性质
- 二叉树的存储
- 二叉树类及操作实现
- 二叉树的遍历

二叉树的遍历

遍历即对结构中每个数据元素进行访问且每个元素只访问一次。它是一种最常见的操作，各种数据结构的基本操作很多都可以以遍历为基础得以实现。

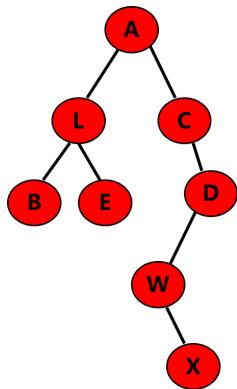
二叉树的遍历

- ▶ **层次遍历**：如果二叉树为空，遍历操作为空。否则，从第一层开始，从上而下，逐层访问每一层结点。对同一层结点，自左向右逐一访问。——思路和建立二叉树相同

基于递归的方法：

- ▶ **前序遍历**：如果二叉树为空，遍历操作为空。否则，先访问根结点，然后前序遍历根的左子树，再前序遍历根的右子树。可简记为：“根左右”。
- ▶ **中序遍历**：如果二叉树为空，遍历操作为空。否则，先中序遍历根的左子树，然后访问根结点，最后中序遍历根的右子树。可简记为：“左根右”。
- ▶ **后序遍历**：如果二叉树为空，遍历操作为空。否则，先后序遍历根的左子树，然后后序遍历根的右子树，最后访问根结点。可简记为：“左右根”。

二叉树的遍历



- 前序：根 → 左子树 → 右子树
前序：A、L、B、E、C、D、W、X
- 中序：左子树 → 根 → 右子树
中序：B、L、E、A、C、W、X、D
- 后序：左子树 → 右子树 → 根
后序：B、E、L、X、W、D、C、A

二叉树遍历的非递归算法思想

遍历即对结构中每个数据元素进行访问且每个元素只访问一次。它是一种最常见的操作，各种数据结构的基本操作很多都可以以遍历为基础得以实现。

分析二叉树的遍历：

二叉链表中每个结点向下有两个叉，即一个结点有两个直接后继结点。

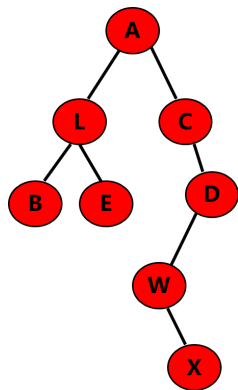
如果先访问了根，下面一个要访问的结点是沿左叉去找？还是沿右叉去找？访问完左叉中的结点是否还能回到其父结点及父结点的右叉上去？二叉链表中结点是不存储父结点地址的，从左叉回到父结点，并不容易。

二叉树遍历的非递归算法思想

- ▶ 二叉树的建立操作，给出了启示，可以利用一个暂存机构完成。
- ▶ 根首先进入暂存机构，然后执行下列操作：只要这个机构里有元素，任意取出一个访问，顺手将其所有孩子结点放入暂存机构。
反复如此，直到暂存机构中没有元素。
可以看出，每个结点都会有唯一的机会进入这个暂存机构，也有唯一的出机构被访问的机会。由此能够达到对每个结点访问且只访问一次的目的。
- ▶ 暂存机构有两种，栈和队列。根据暂存机构的不同，可分为：层次遍历、前序遍历、中序遍历、后序遍历 4 种。

层次遍历

- 按照树的层级从上而下进行遍历，利用队列。
- 层次遍历：
 - 如果二叉树为空，则操作为空；
 - 否则层次上自上而下，每一层从左到右访问每一个结点。



层次遍历：A、L、C、B、E、D、W、X

层次遍历算法分析

层次遍历：如果二叉树为空，遍历操作为空。否则，从第一层开始，从上而下，逐层访问每一层结点。对同一层结点，自左向右逐一访问。

用队列作为辅助工具，让一个队列完全接管结点，由它的进、出队顺序来决定结点的访问次序。

层次遍历算法：

如果二叉树为空，遍历操作为空。否则，将根结点进队，反复循环进行以下操作，直到队空：队首结点出队、访问，如果该结点有左子，左子进队；如果该结点有右子，右子进队。

层次遍历的非递归算法实现

LevelOrder

```
template <class elemType>
void BTree<elemType>::LevelOrder()
//层次遍历二叉树算法的实现。
{   seqQueue<Node<elemType> *> que;
    Node<elemType> *p;

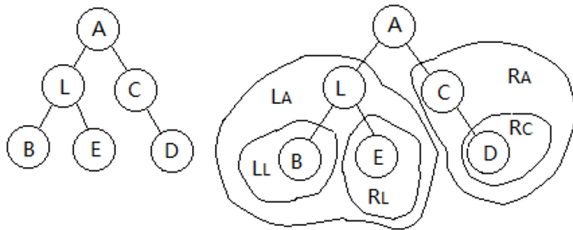
    if (!root) return; //二叉树为空
    que.enqueue(root);
```

```
while (!que.isEmpty())
{
    p = que.front();
    que.dequeue();
    cout << p->data;
    if (p->left) que.enqueue(p->left);
    if (p->right) que.enqueue(p->right);
}
cout << endl;
}
```

每一轮循环都出队访问一个结点，共循环了 n 次，时间复杂度为 $O(n)$ 。

二叉树的前序遍历

- ▶ **前序遍历**：如果二叉树为空，遍历操作为空。否则，先访问根结点，然后前序遍历根的左子树，再前序遍历根的右子树。
- ▶ 右图前序遍历序列：ALBECD



二叉树的前序遍历

- ▶ 对前序遍历的定义是一种递归的形式。
- ▶ 用递归来实现前序遍历非常直观、简单，只需要将定义换成具体的、用高级语言书写的语句就可以了。

PreOrder

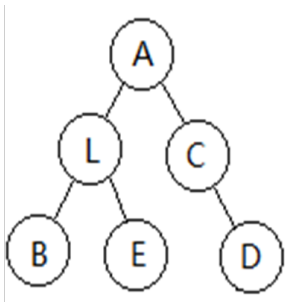
```
template <class elemType>
void BTree<elemType>::PreOrder(Node<elemType> *t)
//前序遍历以t为根二叉树递归算法的实现。
{   if (!t) return;
    cout << t->data;

    PreOrder(t->left);
    PreOrder(t->right);
}
```

时间效率分析：

每次执行两个 return 二选一，每个结点经历一次输出 $t \rightarrow data$ ，每个空链域经历一次 return，共 $n + (2n - (n - 1)) = 2n + 1$ 次。

前序遍历



调用 $\text{PreOrder}(\langle A \rangle)$ 输出 A

调用 $\text{PreOrder}(\langle L \rangle)$ 输出 L

调用 $\text{PreOrder}(\langle B \rangle)$ 输出 B

调用 $\text{PreOrder}(\text{NULL})$ B 的左子

调用 $\text{PreOrder}(\text{NULL})$ B 的右子

调用 $\text{PreOrder}(\langle E \rangle)$ 输出 E

调用 $\text{PreOrder}(\text{NULL})$ E 的左子

调用 $\text{PreOrder}(\text{NULL})$ E 的右子

调用 $\text{PreOrder}(\langle C \rangle)$ 输出 C

调用 $\text{PreOrder}(\text{NULL})$ C 的左子

调用 $\text{PreOrder}(\langle D \rangle)$ 输出 D

调用 $\text{PreOrder}(\text{NULL})$ D 的左子

调用 $\text{PreOrder}(\text{NULL})$ D 的右子

前序遍历的非递归算法

分析：

对于递归，系统内部是用栈来辅助的，现在把栈从幕后拉到前台，显式地使用栈，来消除递归调用。

算法思想：

- ▶ 如果根为空，遍历结束。否则建立一个结点指针栈，先将根进栈。
- ▶ 反复进行以下操作：出栈、访问，如果其有右子，右子压栈；如果其有左子，左子压栈。直到栈空。

前序遍历的非递归算法

思考：

以上算法为什么能遍历？

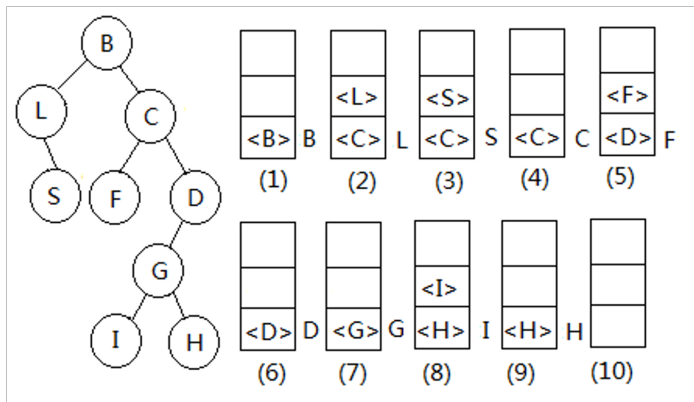
前序遍历的非递归算法

思考：

以上算法为什么能遍历？

- 每个访问到。
- 每个只访问一次。

前序遍历的非递归算法

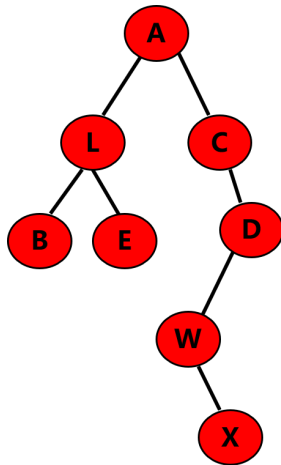


思考:

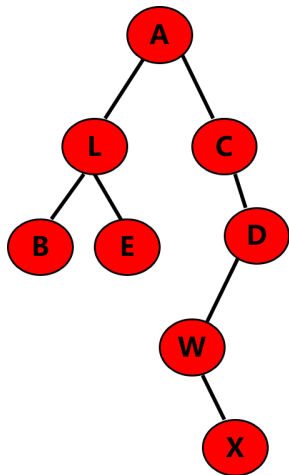
如何由 S 回到 C ? 比较递归和非递归手法的区别

前序遍历的非递归实现

- **前序遍历**：先访问根，然后访问左子树、右子树。
- 设置一个**栈**，保存结点的地址。
- 把根结点地址压栈。重复执行以下过程，直到栈空：
 - ◇ 从栈中弹出一个结点的地址，输出该结点的值；
 - ◇ 如果有**右子**，右子地址入栈；
 - ◇ 如果有**左子**，左子地址入栈。



前序遍历的非递归实现举例



栈 $\longrightarrow$ 输出:

A				
---	--	--	--	--

A

C	L			
---	---	--	--	--

L

C	E	B		
---	---	---	--	--

B

C	E			
---	---	--	--	--

E

C				
---	--	--	--	--

C

D				
---	--	--	--	--

D

W				
---	--	--	--	--

W

X				
---	--	--	--	--

X

1. 根结点进栈;
2. 结点出栈, 访问;
3. 结点的右、左孩子 (非空) 进栈;
4. 反复重复 2、3 步, 直到栈空。

前序遍历的非递归算法实现

PreOrder 非递归

```
template <class elemType>
void BTree<elemType>:: PreOrder()
{
    if (!root) return;
    Node *p;
    seqStack<Node *> s;
    s.push(root);
    while (!s.isEmpty())
    {
        p = s.top(); s.pop();
        cout << p->data;
        if (p->right) s.push(p->right);
        if (p->left) s.push(p->left);
    }
    cout << endl;
}
```

前序遍历的非递归算法分析

分析：

- ▶ 第一层结点，根主动进栈。
- ▶ 根出栈时将其所有子，即第二层结点都带入栈中。类似地，第二层结点出栈时会将第三层结点带入栈中，最后每层结点都有进栈机会。
- ▶ 每个结点都是由其父结点访问时带入栈中，每个结点的父结点唯一，故每个结点进栈的机会只有一次。
- ▶ 每个进栈元素出栈时都会被访问到。

因此，按照此算法能完成遍历的任务。

前序遍历的非递归算法时间复杂度分析

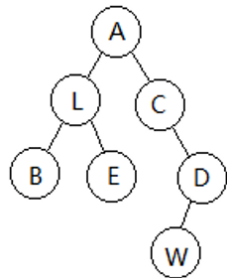
每次循环都从栈中弹出并访问一个结点。

当整个循环结束时，每个结点都被访问且只访问一次，因此循环次数为 n 。

算法的时间复杂度就是 $O(n)$ 。

二叉树的中序遍历

- **中序遍历**：如果二叉树为空，遍历操作为空。否则，先中序遍历根的左子树，访问根结点，然后再中序遍历根的右子树。可简记为：“左根右”。
- 右图中序遍历序列：BLEACWD



深入思考几个问题：

为什么有这种序？递归如何回溯？递归或者栈

二叉树的中序遍历

- ▶ 对中序遍历的定义是一种递归的形式。
- ▶ 用递归来实现中序遍历非常直观、简单，只需要将定义换成具体的、用高级语言书写的语句就可以了。

InOrder

```
template <class elemType>
void BTree<elemType>::InOrder(Node<elemType> *t)
//中序遍历以t为根二叉树递归算法的实现。
{  if (!t) return;

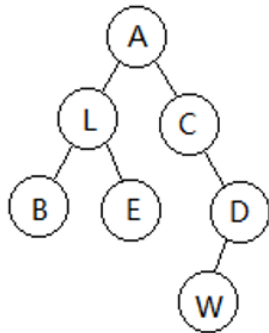
    InOrder(t->left);
    cout << t->data;
    InOrder(t->right);
}
```

第一种中序遍历非递归算法

InOrder

```
template <class elemType>
void Btree<elemType>::inOrder() const
{ if (!root) return;
  seqStack<Node<elemType>*> s; s.push(root);
  Node<elemType> *p; p=root;
  while(p->left) {s.push(p->left); p=p->left;}

  while( !s.isEmpty())
  { p=s.top(); s.pop(); cout<<p->data;
    if ( p->right) { s.push(p->right); p=p->right;
      while(p->left) {s.push(p->left); p=p->
        left;}
    }
  }
}
```



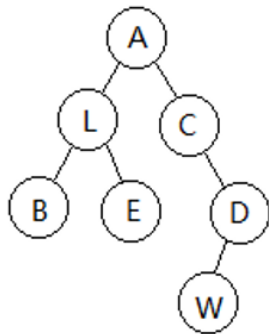
第二种中序遍历的非递归

中序遍历序列：BLEACWD

先根 A 压栈，栈不空，A 出栈，但 A 不能访问，因其左子 L 未访问，为了访问完 L 能再回到 A，故将 A 再次进栈，并将 L 带入栈中。

注意此时栈中 L 在 A 之上，可保证将来弹出访问时，先 L 后 A。当 A 再次出现在栈顶出栈时，因已经考虑过其左子了，故可以直接访问。

为区别其是否已经考虑过左子，可对进栈元素加一个标识，0 表示其未考虑过左子，是第一次进栈；1 表示其考虑过左子，是第二次进栈。

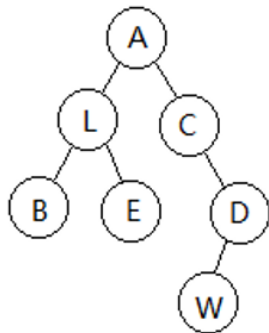


第二种中序遍历的非递归

中序遍历序列：BLEACWD

当结点出栈访问时，如 L 出栈并访问时，说明 L 的左子树已经访问过，按照“左根右”，现在要考虑其右子 E。故当一个结点访问后，如果有右子，将其右子进栈，这样结点 L 的使命才算完成。

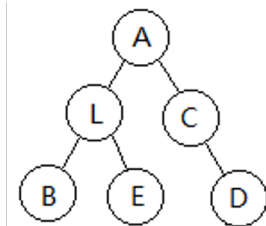
可以看出：根结点为主动进栈，其余结点都是被父结点带入栈中，且带入时，子结点状态都置为 0；出栈并考虑其左子时，才将状态改为 1。



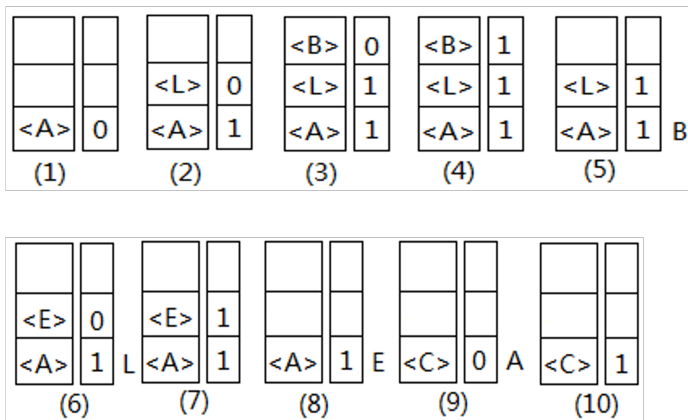
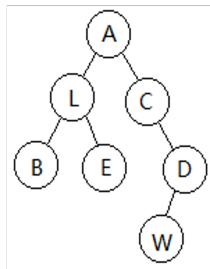
中序遍历的非递归算法

中序遍历序列：BLEACD

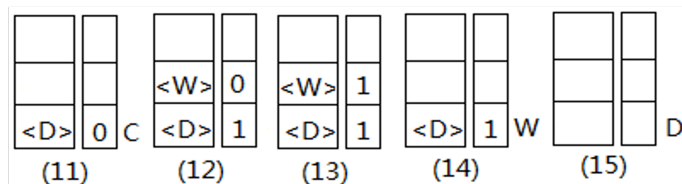
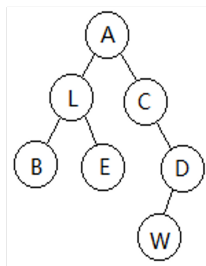
- 如果根为空，遍历结束。否则建立一个结点指针栈，先将根进栈，以 0 作为标志。
- 反复进行以下操作，直到栈空。
出栈，如果出栈元素的标志为 0，改为 1，再次将其压入栈中，如果其有左子，将左子压入栈中，并以 0 作为左子的标志；如果出栈元素的标志为 1，访问，如果其有右子，将右子压入栈中，并以 0 作为右子的标志。



中序遍历的非递归算法中栈的变化



中序遍历的非递归算法中栈的变化



中序遍历的非递归算法实现

InOrder 非递归

```
template <class Type>
void BTree<Type>:: InOrder()
{
    if (!root) return;
    seqStack<Node *> s1;
    seqStack<int> s2;
    Node *p;
    int flag;
    int zero = 0, one = 1;
    p = root;
    s1.push(p); s2.push(zero);
    while (!s1.isEmpty())
    {
        flag = s2.top(); s2.pop();
        p = s1.top(); //读取栈顶元素
        if (flag == 1) //二次出栈，则访问
        {
            s1.pop();
            cout << p->data;
```

```
        if (!p->right) continue;
        //有右子压右子，没有进入下一轮循环
        s1.push(p->right);
        s2.push(zero);
    }
    else //首次出栈，则入栈，标1
    {
        s2.push(one);
        if (p->left) //有左子压左子
        {
            s1.push(p->left);
            s2.push(zero);
        }
    }
    } //end while
    cout << endl;
} //end
```


中序遍历的非递归算法分析

每次循环中都是弹出一个结点，结点标志为 1 时，直接访问；结点标志为 0 时，反手将其再次压栈。

二叉树中的每个结点都进入过栈中，且结点标志为 0 时经历过一次循环操作，标志由 0 变 1；结点标志为 1 时也经历过一次循环操作，直接访问。故对每个结点执行过两次循环操作，总的循环次数为 $2n$ 。

算法的时间复杂度为 $O(n)$ 。

两种中序遍历非递归算法，哪种逻辑上更简单性？

中序遍历的非递归算法分析

每次循环中都是弹出一个结点，结点标志为 1 时，直接访问；结点标志为 0 时，反手将其再次压栈。

二叉树中的每个结点都进入过栈中，且结点标志为 0 时经历过一次循环操作，标志由 0 变 1；结点标志为 1 时也经历过一次循环操作，直接访问。故对每个结点执行过两次循环操作，总的循环次数为 $2n$ 。

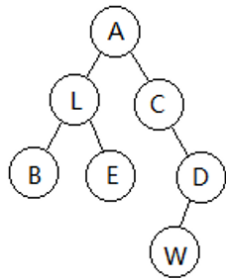
算法的时间复杂度为 $O(n)$ 。

两种中序遍历非递归算法，哪种逻辑上更简单性？

后者，每次操作只涉及到局部片段即父子信息，不涉及到更多子孙。时间效率上有何区别？

二叉树的后序遍历

- **后序遍历**：如果二叉树为空，遍历操作为空。否则，先后序遍历根的左子树，再后序遍历根的右子树，最后访问根结点。可简记为：“左右根”。
- 右图中序遍历序列：BELWDCA



二叉树的后序遍历

- ▶ 对后序遍历的定义是一种递归的形式。
- ▶ 用递归来实现后序遍历非常直观、简单，只需要将定义换成具体的、用高级语言书写的语句就可以了。

InOrder

```
template <class elemType>
void BTree<elemType>::PostOrder(Node<elemType> *t)
//后序遍历以t为根二叉树递归算法的实现。
{
    if (!t) return;
    PostOrder(t->left);
    PostOrder(t->right);
    cout << t->data;
}
```

后序遍历的非递归算法分析

- ▶ 按照中序遍历的非递归算法类似的思路，后序遍历的非递归算法中结点在标志栈中拥有更多的状态：0 表示结点首次进栈，1 表示结点出栈过一次即考虑过左子，2 表示结点出栈过两次即考虑过右子。
- ▶ 栈中弹出的结点，如果标志位为 0，将其标志改为 1 并反手将该结点再次压入栈中，如果该结点有左子，随即将其左子压入栈中；栈中弹出的结点，如果标志位为 1，将其标志改为 2 并反手将该结点再次压入栈中，如果该结点有右子，随即将其右子压入栈中；栈中弹出的结点，如果标志位为 2，就可以直接进行访问了。
- ▶ 在压栈过程中，只有根结点是主动压入栈中，压栈时标志置为 0，其他结点都是在父结点弹出栈时，被首次压入栈中的，这些首次压入栈中的结点，标志置为 0。
- ▶ 非递归算法中，循环的次数变为 $3n$ ，算法的时间复杂度依然为 $O(n)$ 。

后序遍历的非递归算法实现

PostOrder 非递归

```
template <class elemType>
void BTree<elemType>:: PostOrder()
//后序遍历的非递归算法实现。
{
    if (!root) return;
    Node *p;
    seqStack<Node *> s1;
    seqStack<int> s2;
    int zero = 0, one = 1, two = 2;
    int flag;
    s1.push(root);
    s2.push(zero);
    while (!s1.isEmpty())
    {
        flag = s2.top(); s2.pop();
        p = s1.top();
        switch(flag) {
            case 2:
```

```
            s1.pop();
            cout << p->data;
            break;
            case 1:
                s2.push(two);
                if (p->right) {
                    s1.push(p->right);
                    s2.push(zero);}
                break;
            case 0:
                s2.push(one);
                if (p->left) {
                    s1.push(p->left);
                    s2.push(zero);}
                break;
        }; //switch
    } //while
    cout << endl;}
```

后序遍历的非递归算法实现

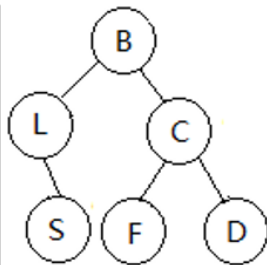
每次循环是一个结点的 0、1、2 状态三选一；

每个结点都要经历 0、1、2；

n 个结点共 $3n$ 次

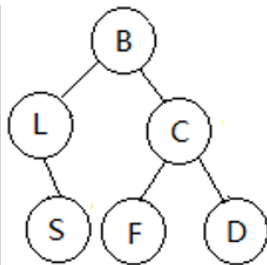
二叉树的层次遍历

- ▶ 二叉树的层次遍历为从上到下逐层访问，每一层从左到右逐个访问每个结点。
- ▶ 右图的一棵二叉树的层次遍历序列为 B、L、C、S、F、D。
- ▶ 算法思路参照 createTree 函数实现



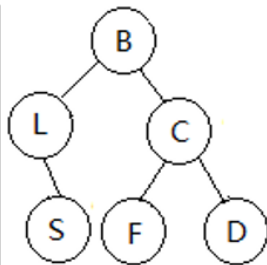
二叉树的后序遍历

- ▶ 上面的方法，副产品是可方便地求出所有祖先。
- ▶ 还有其它方法吗？
flag 0->1 右子和左子同时进栈？



二叉树的后序遍历

- ▶ 查找某个元素、求结点个数
- ▶ 求结点层次、树的高度
- ▶ 求某个结点的子孙结点
- ▶ 求某个结点的祖先结点



- ▶ 树
- ▶ 二叉树
- ▶ 遍历序列确定二叉树
- ▶ 最优二叉树
- ▶ 树和森林
- ▶ 二叉线索树 *
- ▶ 表达式树 *
- ▶ 等价关系 *
- ▶ 优先级队列

遍历序列确定二叉树（也称二叉树的重构）

- ▶ 已知一棵**完全二叉树**的层次遍历，能唯一确定这个完全二叉树。
- ▶ 已知一棵**满二叉树**的前序、中序、后序遍历之一，能唯一确定这棵满二叉树。
- ▶ 已知一个**一般二叉树**的前序、中序、后序遍历之一，是不能唯一确定这棵二叉树的。

根据遍历序列确定二叉树

- 已知一个二叉树的前序、中序、后序遍历之一，**不能**唯一确定这棵二叉树。
- 已知一个二叉树的**前序** + **后序**遍历，**不能**唯一确定这棵二叉树。



前序: AB

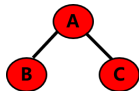
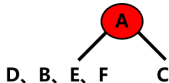
后序: BA

- 已知二叉树的**前序** + **中序**遍历，**能**唯一确定这棵二叉树。
- 已知二叉树的**后序** + **中序**遍历，**能**唯一确定这棵二叉树。

前序 + 中序唯一确定一棵二叉树

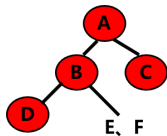
- 前序: A、B、D、E、F、C

- 中序: D、B、E、F、A、C



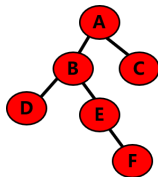
- 前序: B、D、E、F

- 中序: D、B、E、F



- 前序: E、F

- 中序: E、F



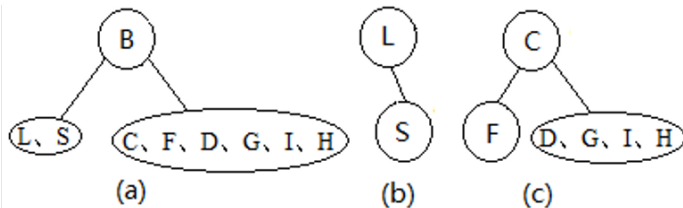
信息互补:

- 前序的首位置是根;
- 中序按照根分成左右两部;
- 重复上述过程。

已知一棵二叉树的前序和中序序列

前序序列: B、L、S、C、F、D、G、I、H

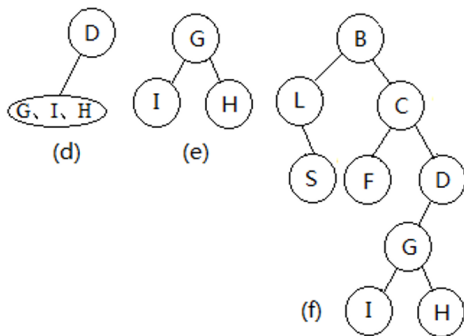
中序序列: L、S、B、F、C、I、G、H、D



已知一棵二叉树的前序和中序序列

前序序列: B、L、S、C、F、D、G、I、H

中序序列: L、S、B、F、C、I、G、H、D

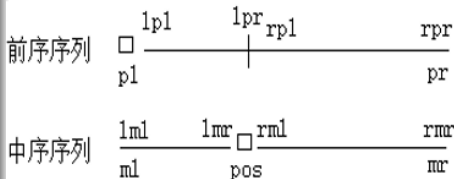


- ▶ 两个序列分别在前序数组和中序数组中。
- ▶ 由前序数组定出根结点值后，在中序数组找到根结点值所在的位置，由此找到了前序和中序序列中根的左子树下标范围，同样也找到了右子树的下标范围。
- ▶ 根据根结点的值创建根结点空间，并分别利用两个数组和其左子树下标范围、右子树下标范围递归确定其左、右子树，将左右子树的根地址写入根结点的左右孩子指针中。

前序+中序唯一确定一棵二叉树

BuildTree

```
template <class elemType>
typename BTree<elemType>:: Node *BTree<elemType>::
buildTree(elemType pre[], int pl, int pr,
          elemType mid[], int ml, int mr)
//pre数组存储了前序遍历序列, pl为序列左边界下标,
//pr为序列右边界下标。
//mid数组存储了中序遍历序列, ml为序列左边界下标,
//mr为序列右边界下标。
{
    Node *p, *leftRoot, *rightRoot;
    int i, pos, num;
    int lpl, lpr, lml, lmr; //左子树中前序的左右边
    //界、中序的左右边界
    int rpl, rpr, rml, rmr; //右子树中前序的左右边
    //界、中序的左右边界
```

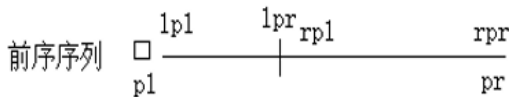


前序+中序唯一确定一棵二叉树

BuildTree

```
if (pl > pr) return NULL;
p = new Node(pre[pl]); //找到子树的根并
    创建结点
if (!root) root = p;
//找根在中序中的位置和左子树中结点个数
for (i = ml; i <= mr; i++)
    if (mid[i] == pre[pl]) break;
pos = i; //子树根在中序中的下标
num = pos - ml; //子树根的左子树中结点的个
    数
//找左子树的前序、中序序列下标范围
lpl = pl + 1; lpr = pl + num;
```

```
lml = ml; lmr = pos - 1;
leftRoot = buildTree(pre, lpl, lpr, mid,
    lml, lmr);
//找右子树的前序、中序序列下标范围
rpl = pl + num + 1; rpr = pr;
rml = pos + 1; rmr = mr;
rightRoot = buildTree(pre, rpl, rpr, mid,
    rml, rmr);
p->left = leftRoot; p->right = rightRoot;
return p;
}
```



- ▶ 树
- ▶ 二叉树
- ▶ 遍历序列确定二叉树
- ▶ 最优二叉树
- ▶ 树和森林
- ▶ 二叉线索树 *
- ▶ 表达式树 *
- ▶ 等价关系 *
- ▶ 优先级队列

最优二叉树

二叉树中任意两个结点间的**路径长度**为其路径上的分支总数。

二叉树的路径长度为根到树中各个结点的路径长度之和。

特别地：如果二叉树中**叶子结点**上带有权值，**二叉树的加权路径长度**特指从根结点到各个叶子结点路径上的分支数乘以该叶子的权值之和。记为：

$$WPL = \sum_{k=1}^n w_k L_k$$

WPL 表示加权路径长度、n 为叶子结点的个数、 w_k 为第 k 个叶子的权值、 L_k 为根到第 k 个叶子的路径长度。

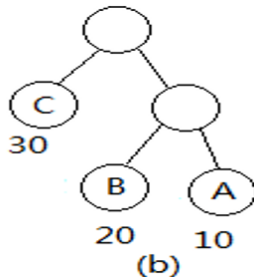
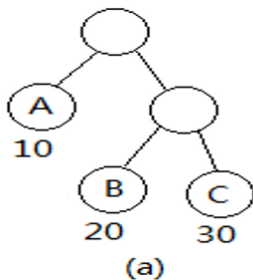
最优二叉树

对同一组带权的叶子结点 (A, 10), (B, 20), (C, 30),

图 (a) 中, $WPL=10*1+20*2+30*2=110$,

图 (b) 中, $WPL=30*1+20*2+10*2=90$ 。

即不同的二叉树的形态会使得其带权路径长度不同。

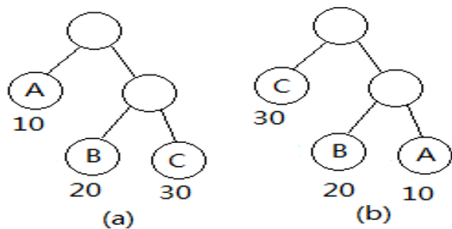


最优二叉树

当一组叶子的权值确定后，假设分别为 $w_1, w_2, \dots, w_n$,

将这些叶子以何种策略挂在一棵二叉树上，或者说这棵二叉树是怎样的形态，才能使其带权路径 WPL 达到最小？

把能使 WPL 达到最小的二叉树，称为**最优二叉树**。

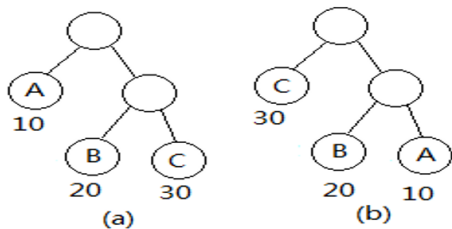


最优二叉树

当一组叶子的权值确定后，假设分别为 $w_1, w_2, \dots, w_n$,

将这些叶子以何种策略挂在一棵二叉树上，或者说这棵二叉树是怎样的形态，才能使其带权路径 WPL 达到最小？

把能使 WPL 达到最小的二叉树，称为**最优二叉树**。



a) 110 b) 90

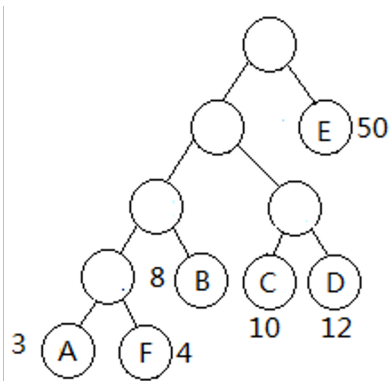
策略：权值越大，越靠近根

最优二叉树

一组带权结点

$U=(A, 3), (B, 8), (C, 10), (D, 12),$
 $(E, 50), (F, 4),$

右侧为其一棵最优二叉树。

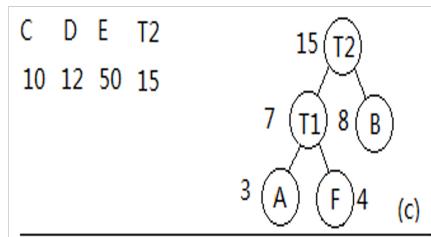
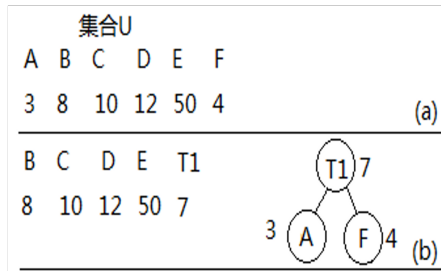


哈夫曼算法

- ▶ 哈夫曼算法就是利用了权值越大，越靠近根的思想，逐步比较结点的权值，构造出了哈夫曼树，哈夫曼树是一棵最优二叉树。
- ▶ 哈夫曼算法的具体步骤为：
对于给定的一个带权结点集合 U
 - ① 如果 U 中只有一个结点，操作结束，否则转向 2)。
 - ② 在集合中选取两个权值最小的结点 x 、 y ，构造一个新的结点 z 。新结点 z 的权值为结点 x 、 y 的权值之和。在集合 U 中删除结点 x 和 y 并加入新结点 z ，然后转向 1)。

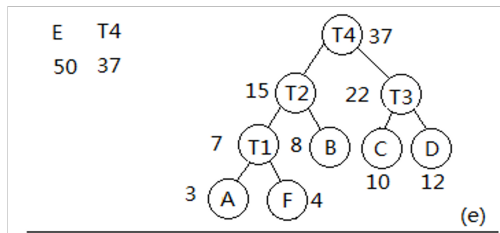
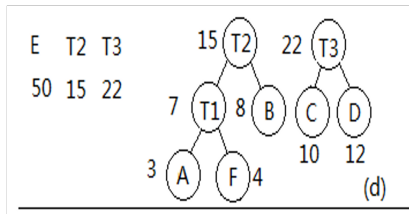
哈夫曼算法的构造示例

对一组带权结点 $U=(A, 3), (B, 8), (C, 10), (D, 12), (E, 50), (F, 4)$, 按照哈夫曼算法构造一棵最优二叉树。



哈夫曼算法的构造示例

对一组带权结点 $U=(A, 3), (B, 8), (C, 10), (D, 12), (E, 50), (F, 4)$, 按照哈夫曼算法构造一棵最优二叉树。

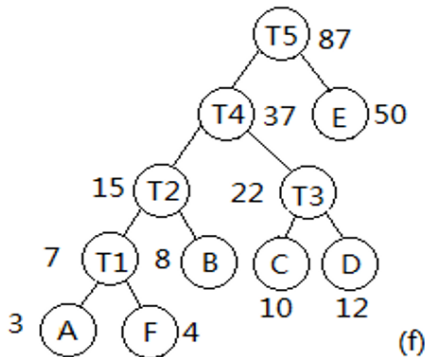


哈夫曼算法的构造示例

对一组带权结点 $U=(A, 3), (B, 8), (C, 10), (D, 12), (E, 50), (F, 4)$, 按照哈夫曼算法构造一棵最优二叉树。

T5

87



哈夫曼算法构造出的二叉树一定是一个最优二叉树吗？

定理 4-1: 设 T 为带权 $w_1 \leq w_2 \leq \dots \leq w_t$ 的一组结点集合构成的最优二叉树，则

- 1) 带权 w_1, w_2 的叶子 v_{w_1}, v_{w_2} 是兄弟。
- 2) 以叶子 v_{w_1}, v_{w_2} 为儿子结点的内部结点，其路径长度最长。

定理 4-2: 设 T 为带权 $w_1 \leq w_2 \leq \dots \leq w_t$ 的一组结点集合构成的最优二叉树，若将以带权 w_1 和 w_2 的叶子为儿子的内部结点改为带权 $w_1 + w_2$ 的叶子，得到一棵新树 T_1 ，则 T_1 也是最优二叉树。

两个定理循环往复，正是哈夫曼算法正确性的说明。

哈夫曼算法的实现

二叉树用顺序存储法：用一个数组存储哈夫曼结点。

哈夫曼结点（叶子、中间结点） **HuffmanNode**：

包含 5 个字段：

data 为结点的值

weight 为结点的权值

parent 为结点的父结点地址（下标）

left 和 right 为结点的左、右孩子的地址（下标）。

特殊地：数组的 0 下标分量空出来不用，从下标为 1 的数组分量开始存储数据。

哈夫曼算法的实现

假定树中叶子结点有 n 个, 中间结点都是由两个结点构造而成的, 度均为 2。按照 $n_2 = n_0 - 1$ 性质, 最优二叉树中结点总数为:

$$n + n - 1 = 2n - 1$$

开辟动态数组时, 考虑到 0 下标分量不用, 需要为数组申请 $2n$ 个连续的结点空间。

初始时, 数组中只有叶子结点, 其 parent、left、right 都设置为 0。

叶子结点在数组中从后往前依次存储, 前面空余的 $n-1$ 个数组分量作为存储即将构造的中间结点用。

哈夫曼算法结果

对一组带权结点 $U=(A, 3), (B, 8), (C, 10), (D, 12), (E, 50), (F, 4)$, 按照哈夫曼算法构造了一棵最优二叉树。

index	1	2	3	4	5	6	7	8	9	10	11
data						F	E	D	C	B	A
weight						4	50	12	10	8	3
parent						0	0	0	0	0	0
left						0	0	0	0	0	0
right						0	0	0	0	0	0

哈夫曼算法结果

对一组带权结点 $U=(A, 3), (B, 8), (C, 10), (D, 12), (E, 50), (F, 4)$, 按照哈夫曼算法构造了一棵最优二叉树。

index	1	2	3	4	5	6	7	8	9	10	11
data						F	E	D	C	B	A
weight					7	4	50	12	10	8	3
parent					0	5	0	0	0	0	5
left					11	0	0	0	0	0	0
right					6	0	0	0	0	0	0

哈夫曼算法结果

对一组带权结点 $U=(A, 3), (B, 8), (C, 10), (D, 12), (E, 50), (F, 4)$, 按照哈夫曼算法构造了一棵最优二叉树。

index	1	2	3	4	5	6	7	8	9	10	11
data						F	E	D	C	B	A
weight				15	7	4	50	12	10	8	3
parent				0	4	5	0	0	0	4	5
left				5	11	0	0	0	0	0	0
right				10	6	0	0	0	0	0	0

哈夫曼算法结果

对一组带权结点 $U=(A, 3), (B, 8), (C, 10), (D, 12), (E, 50), (F, 4)$, 按照哈夫曼算法构造了一棵最优二叉树。

index	1	2	3	4	5	6	7	8	9	10	11
data						F	E	D	C	B	A
weight	87	37	22	15	7	4	50	12	10	8	3
parent	0	1	2	2	4	5	1	3	3	4	5
left	2	4	9	5	11	0	0	0	0	0	0
right	7	3	8	10	6	0	0	0	0	0	0

哈夫曼算法的实现代码 huffman.h

HuffmanNode

//存储 Huffman 结点

```
template <class elemType>
struct HuffmanNode
{
    elemType data;
    double weight;
    int parent;
    int left, right;
};
```

//在所有父亲为0的元素中找最小值的下标

```
template <class elemType>
int minIndex(HuffmanNode<elemType> Bt[], int
k, int m)
{
    int i, min, minWeight = 9999; //用一个不可
    能且很大的权值
    for (i=m-1; i>k; i--)
    {
        if ((Bt[i].parent==0)&&(Bt[i].weight
        < minWeight))
        {
            min = i;      minWeight = Bt[min].
            weight;      }
    }
    return min;
}
```

可换成优先级队列，两次出队取到最小和次小值。

哈夫曼算法的实现代码 huffman.h

BestBinaryTree

```
template <class elemType>
HuffmanNode<elemType> *BestBinaryTree (
    elemType a[],
    double w[], int n)
{
    HuffmanNode<elemType> *BBTree;
    int first_min, second_min; //
    int m=n*2; //共 $2n-1$ 个结点, 下标为0处不放
               结点
    int i, j;
```

```
//在数组中从后往前存储叶子结点
BBTree = new HuffmanNode<elemType>[m];
for (j=0; j<n; j++)
{
    i = m-1-j;
    BBTree[i].data = a[j];      BBTree[i].
    weight = w[j];
    BBTree[i].parent = 0;      BBTree[i].
    left = 0;
    BBTree[i].right = 0;
}
i = n-1; // i is the position which is
          ready for the first new node
```

哈夫曼算法的实现代码 huffman.h

BestBinaryTree

```
while (i!=0) //数组左侧尚有未用空间，即新创建的结点个数还不足
{
    first_min = minIndex(BBTree, i, m);
    BBTree[first_min].parent = i;
    second_min = minIndex(BBTree, i, m);
    BBTree[second_min].parent = i;

    BBTree[i].weight = BBTree[first_min].weight + BBTree[second_min].weight;
    BBTree[i].parent = 0;
    BBTree[i].left = first_min; BBTree[i].right = second_min;
    i--;
}
return BBTree;
}
```

哈夫曼算法的时间复杂度分析

为 n 个叶子结点设置初始状态，时间消耗 $O(n)$ 。执行了 $n-1$ 次创建新的中间结点操作，每次创建要在所有元素（元素个数在 n 和 $2n$ 之间）中两次去找权值最小的元素，时间耗费为 $2n$ ，算法总的时间复杂度为 $O(n^2)$ 。

可改进：

- ① 扫描一遍同时得出最小和次小值，算法总时效仍为 $O(n^2)$ 。
- ② 利用优先级队列两次出队得最小和次小值，一次入队进新构建的中间结点，算法总时效仍为 $O(n\log_2 n)$ 。

哈夫曼编码

一般来说，字符编码采用等长策略，即每个字符的二进制编码长度一样。在通讯业务中，希望传送的编码越短越好。尤其是对于高频传送的字符，希望其编码尽可能短，而低频字，因为不常用，可以比较长些。

前缀编码：在一个字符集中，字符的编码可以不等长，但任何一个字符的编码不得是另外一个编码的前缀。

如 a 编码 110，其前缀则 1、11 不得是任何字符编码，同样以 110 为前缀的编码也不得出现。哈夫曼编码是一种前缀编码。（保证了译码没有二义性）

如现在传送业务中只有 4 种不同字符 A、B、C、D，用等长编码可使用长度为 2 的编码，它们可以分别是 00、01、10、11。

如果其中字符 A 的使用频率很高，而字符 D 的使用频率很低，就可以采用将字符 D 的编码位数拉长，换取 A 的编码位数缩短的策略来缩短总的字符传送量。

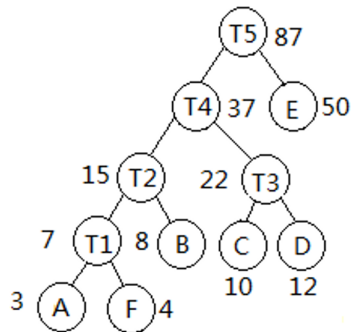
按照这个思路，编码由等长变为了不等长，而哈夫曼树就可以用来构造这样的不等长编码。

哈夫曼编码

用等长编码，为表示 6 个元素，需要用三位编码。即如果待传输的短文中有 n 个字符，就需要 $3n$ 位才能代表这篇短文。

使用了哈夫曼编码后， n 个字符所需要的总位数为：

$$n * (4 * 3 / 87 + 4 * 4 / 87 + 3 * 8 / 87 + 3 * 10 / 87 + 3 * 12 / 87 + 1 * 50 / 87) = 168n / 87$$
，显然它连 $2n$ 都不到，这样就大大减少了通讯中的传输量。



求哈夫曼编码的算法

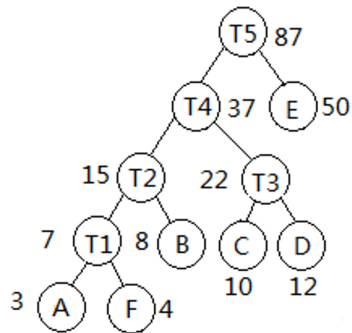
解释了为什么结构中有 parent 字段

对哈夫曼树中任意一个叶子结点求其哈夫曼编码，将该叶子结点设为当前结点，顺着当前结点往上追溯到父亲结点。

如果当前结点是父亲的左子就输出一个 0，如果当前结点是父亲的右子就输出一个 1，之后设父亲为当前结点，反复进行以上操作，直到当前结点为根结点。

在这个过程中输出的 0、1 序列的逆序即其哈夫曼编码。

如图中 F 的编码为：0001



求哈夫曼编码的算法实现

- ▶ 函数的参数 BBTre 数组，保存了哈夫曼树的结构。
- ▶ 函数用了一个栈保存输出的 0、1 字符序列，对一个叶子，在其每一步向上追溯父结点的过程中，将输出的 0 或 1 进栈，当追溯工作因达到根结点而结束时，将栈中元素弹栈，即得到哈夫曼编码。
- ▶ 数组 HFCode 用来存储获得的所有哈夫曼编码，数组的每个分量指向了一个字符串，它是某个叶子结点的哈夫曼编码。
- ▶ 算法实现首先从数组尾部开始，逐步为每一个叶子求其哈夫曼编码，共有 n 个叶子。

求哈夫曼编码的算法实现

HuffmanCode

```
template <class elemType>
char ** HuffmanCode ( HuffmanNode<elemType> BBTree[ ], int n )
//n为待编码元素的个数, BBTree数组为Huffman树, 数组长度为2n
{
    seqStack<char> s;
    char **HFCode;
    char zero = '0', one = '1';
    int m, i, j, parent, child;
    //为HFCode创建空间
    HFCode = new char* [n];
    for (i=0; i<n; i++)
        HFCode[i] = new char[n+1]; //每位元素编码最长n-1位, +1为n=1时储备
        m=2*n; //BBTree数组长度
    if (n==0) return HFCode; //没有元素
    if (n==1) //元素个数为1
    {
        HFCode[0][0] = '0', HFCode[0][1] = '\0';
        return HFCode;
    }
}
```

求哈夫曼编码的算法实现

HuffmanCode

```
for (i=m-1; i>=n; i--)
{
    child=i;
    parent = BBTree[child].parent;
    while (parent!=0)
    {
        if (BBTree[parent].left==child)
            s.push(zero);
        else
            s.push(one);
        child = parent;
        parent = BBTree[parent].parent;
    }
```

//在数组中从后往前存储叶子结点

```
    j=0;
    while (!s.isEmpty())
    {
        HFCode[m-i-1][j] = s.top();
        s.pop();
        j++;
    }
    HFCode[m-i-1][j] = '\0';
}
return HFCode;
```

求哈夫曼编码的算法时间复杂度分析

该方法为对哈夫曼树自下而上访问。

算法包含了两重循环，外循环次数为叶子结点的个数 n ，内循环串行地做了两件事：一个是从叶子逐步追溯到根获取哈夫曼编码的逆序，一个是逐步弹栈获取哈夫曼编码。

内循环的两个操作消耗的时间都最多是哈夫曼树的高度，而哈夫曼树的形态、高度取决于这组字符的频度分布。

最好时，哈夫曼可能达到的树高是 $\log_2 n$ ；最差时，哈夫曼树的树高会达到 n ，因此求哈夫曼编码算法的时间复杂度为 $O(n^2)$ 。

求哈夫曼编码的算法改进分析

如果对该哈夫曼树自上而下，即从 `root` (1 下标) 开始利用遍历的方式进行访问，如可利用层次遍历（利用一个队列），为每一个访问的结点求取编码。

求取时，根编码为空串，后面每个结点被父结点带入队列时，左孩子由父结点编码尾部追加 ‘0’ 构成其编码，右孩子由父结点编码尾部追加 ‘1’ 构成其编码。

访问某结点时，如果是叶子结点，存储其编码即可。

求哈夫曼编码的算法改进分析

层次遍历访问了 $2n-1$ 个结点，时间效率为： $O(n)$ 。

对每个父结点字符串追加 '0' 或者 '1' 的效率：

如果利用普通字符数组存储，时间效率为串的长度即树高。哈夫曼树高最好时是 $\log_2 n$ ；最差时是 n 。算法总时间度仍为 $O(n^2)$ 。

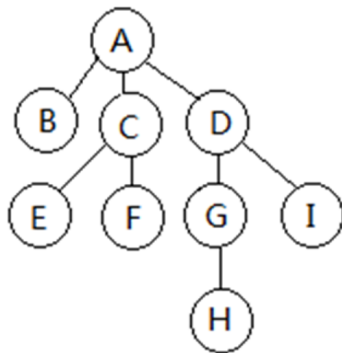
如果利用 `string` 类对象存储字符串，类中有 `length` 属性的支持，追加字符的效率为 $O(1)$ 。算法总时间复杂度为 $O(n)$ 。

- ▶ 树
- ▶ 二叉树
- ▶ 遍历序列确定二叉树
- ▶ 最优二叉树
- ▶ 树和森林
- ▶ 二叉线索树 *
- ▶ 表达式树 *
- ▶ 等价关系 *
- ▶ 优先级队列

树和森林的存储方法

- ▶ 双亲表示法：顺序存储
- ▶ 孩子兄弟法：二叉树存储

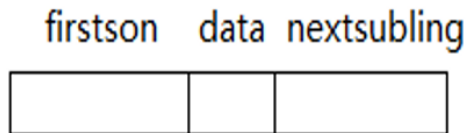
常用孩子兄弟表示法，
二叉树基本操作都可利用上。



(a) 一棵树

孩子兄弟表示法

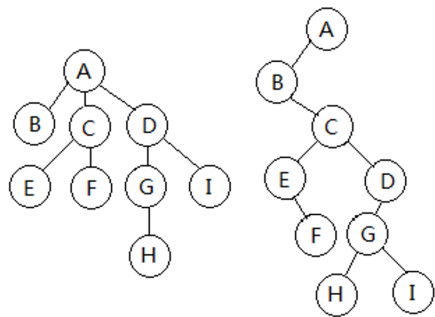
每个结点除了保存数据，还保存了该结点的最大孩子结点地址和最大弟弟结点的地址。



`data` 字段保存了结点数据、`firstchild` 字段保存了最大孩子结点的地址、`nextsibling` 字段保存了最大弟弟结点的地址。

以下为了方便，有时称 `firstchild` 为结点的左分支、左子或左手，称 `nextsibling` 为右分支、右子或右手。

树的孩子兄弟表示法示例



(a) 一棵树

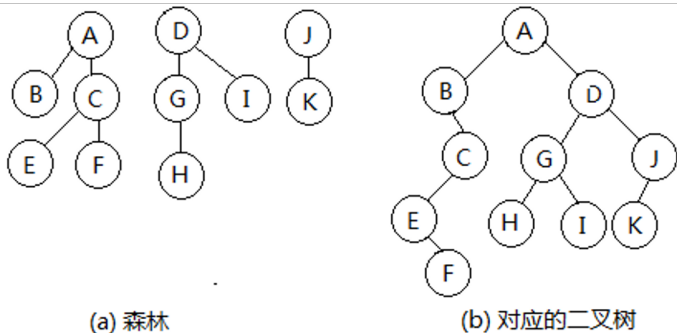
(b) 对应的二叉树

思考：

- ① 树中每个结点都在这棵二叉树上吗？
- ② 树中每个结点在这棵二叉树上只出现一次吗？
- ③ 这棵二叉树唯一吗？即树和二叉树一一对应吗？

森林的孩子兄弟表示法

每棵树按照孩子兄弟法存储后，再将每棵树的根视作兄弟，一个接一个地链在最小哥哥的右手上。

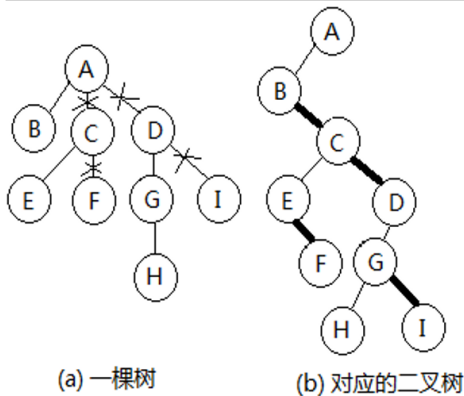


森林和二叉树一一对应

树转化为对应二叉树的方法

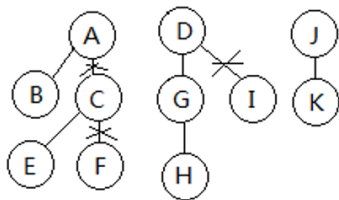
树的根就是二叉树的根。

对树中每个结点，保留其到最大孩子的分支作为左分支，对其余孩子删除其到父结点的分支，逐个降级，增加其左侧最小哥哥到它的右分支，将它链到最小哥哥结点的右分支上去。

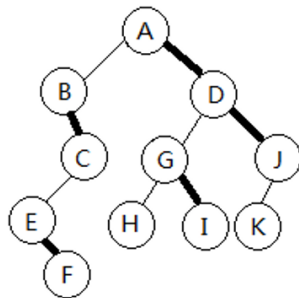


森林转化为对应二叉树的方法

- ▶ 将森林中的每棵树转换为对应的二叉树，每棵二叉树的根就是它对应树的根。
 - ▶ 将每棵树的根看作兄弟，即二叉树的根为兄弟。
 - ▶ 将第一棵二叉树的根作为森林对应的二叉树的根。
- 其余二叉树的根由其最小的哥哥结点用 `nextsibling` 字段链接。



(a) 森林



(b) 对应的二叉树

将二叉树转换为树或森林

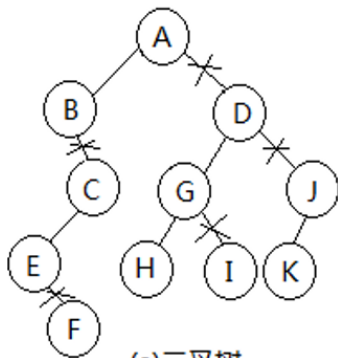
树、森林和对应的二叉树之间的关系是一一对应的吗？

严格说是有序树、有序森林对应的二叉树是唯一的。

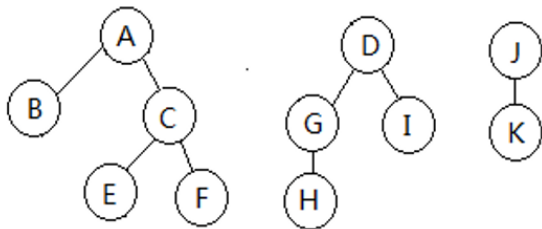
转换方法：

- ▶ 二叉树的根即第一棵树的根。
- ▶ 断开每个结点的 `nestsibling` 发出的分支，将右分支上结点上移至连续右分支的最上层。如果该上层结点有父结点，右分支结点作为该父结点的次子建立其间的分支；如果该上层结点无父结点，右分支结点作为一棵新树的根结点。

将二叉树转换为树或森林



(a) 二叉树



(b) 对应的森林

树的遍历

树的遍历有**两种方式**：

先根遍历（或称先序遍历）和后根遍历（或称后序遍历）。

- ▶ 先根遍历访问完根结点后再逐个从左到右先根遍历其子树
- ▶ 后根遍历从左到右逐个后根遍历完其所有子树后再访问根

根有多个孩子，显然无法定义中根遍历。

用递归的方式定义树的先根遍历和后根遍历

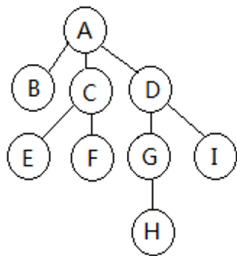
先根遍历：

- ① 如果根结点为空，遍历操作为空，否则访问根结点。
- ② 从左到右，逐个先根遍历以根结点的孩子为根的子树。

后根遍历：

- ① 如果根结点为空，遍历操作为空，否则从左到右，逐个后根遍历以根结点的孩子为根的子树。
- ② 访问根结点。

示例

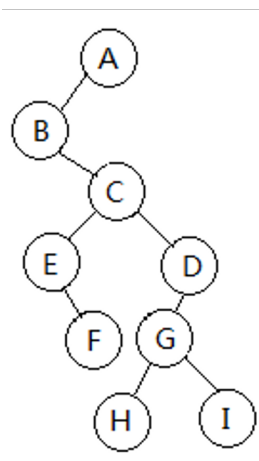


先根遍历: A、B、C、E、F、D、G、H、I

是对应二叉树的前序遍历

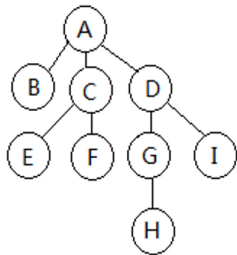
后根遍历: B、E、F、C、H、G、I、D、A

是对应二叉树的中序遍历



示例

树的先根遍历就是对应二叉树的先序遍历。树的后根遍历就是对应二叉树的中序遍历。

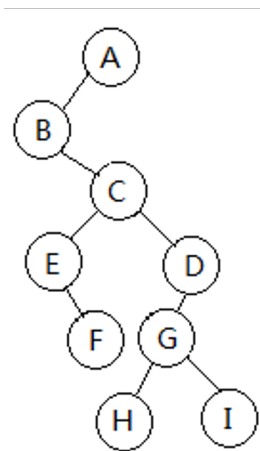


先根遍历: A、B、C、E、F、D、G、H、I

是对应二叉树的前序遍历

后根遍历: B、E、F、C、H、G、I、D、A

是对应二叉树的中序遍历



森林的遍历：先序遍历和中序遍历

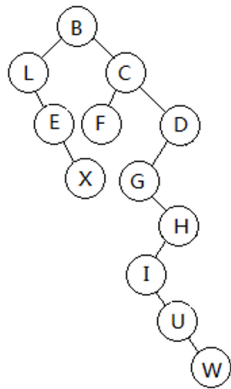
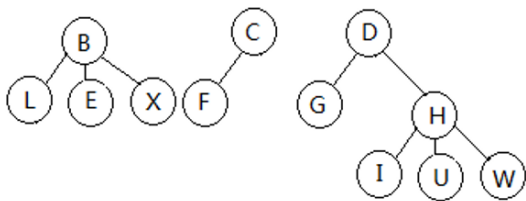
先序遍历：

- ① 如果森林为空，遍历操作为空。
- ② 访问第一棵树的根结点。
- ③ 先序访问第一棵树中根结点的所有子树形成的森林。
- ④ 从左到右同样方式依次访问第二棵、第三棵、直至所有树。

中序遍历：（相当于从左到右，对每棵树后根遍历）

- ① 如果森林为空，遍历操作为空。
- ② 中序遍历第一棵树中根的子树形成的森林。
- ③ 访问第一棵树的根结点。
- ④ 从左到右同样方式依次访问第二棵、第三棵、直至所有树。

示例



先序遍历: B、L、E、X、C、F、D、G、H、I、U、W

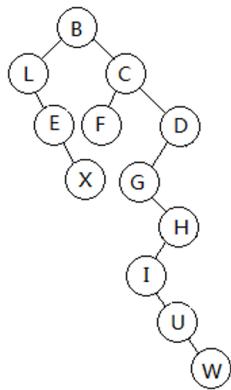
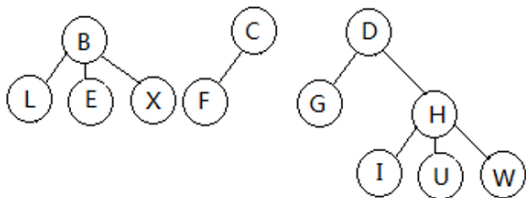
是对应二叉树的前序遍历

中序遍历: L、E、X、B、F、C、G、I、U、W、H、D

是对应二叉树的中序遍历

示例

森林的先序遍历就是对应二叉树的先序遍历。森林的中序遍历就是对应二叉树的中序遍历。



先序遍历: B、L、E、X、C、F、D、G、H、I、U、W

是对应二叉树的前序遍历

中序遍历: L、E、X、B、F、C、G、I、U、W、H、D

是对应二叉树的中序遍历

树和森林的基本操作分析

树形结构，遍历操作是基础，既然对树和森林的遍历都可以转换为对应二叉树的遍历，只要掌握了二叉树的基本操作和算法思路，就能找到树和森林中问题的解决方法。

如：在内存中有一个用二叉树表示的现实生活中的一棵树。

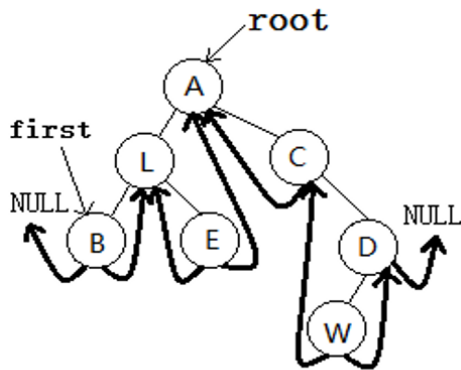
- ① 如何求树的高度？这个问题可以利用二叉树的前序遍历，访问一个结点时，左子的层次数为父结点层次加一，右子的层次数和父结点的一样的方法来处理。
- ② 求森林中树有几棵，可以在二叉树中顺着根，一路右子计数过去，便可解决。

- ▶ 树
- ▶ 二叉树
- ▶ 遍历序列确定二叉树
- ▶ 最优二叉树
- ▶ 树和森林
- ▶ 二叉线索树 *
- ▶ 表达式树 *
- ▶ 等价关系 *
- ▶ 优先级队列

二叉线索树

- ▶ 二叉树中必有 $n+1$ 个左右指针域是空的。
- ▶ 空指针域利用起来：
 - 如果结点的左指针域为空，就存储某种遍历序列中该结点的直接前驱结点地址。
 - 如果结点的右指针域为空，就存储某种遍历序列中该结点的直接后继结点地址。
- ▶ 在空指针域上加载了以上遍历线索的二叉树称线索树。
- ▶ 线索树有：前序、中序、后序线索树三种，中序线索树常见。
- ▶ 在中序线索树上实现中序遍历操作可摆脱对栈的依赖。
- ▶ 有意思的是：在中序线索树上也可脱离栈进行前序遍历。

二叉树的中序遍历



中序遍历序列: BLEACWD

- ▶ 粗线条为增加的线索，左子指向直接前驱，右子指向直接后继。
- ▶ 线索树中结点结构，除了 data、left、right 字段，还多了指针 leftFlag 和 rightFlag，用以区分 left 和 right 指向了孩子还是线索。
- ▶ 线索树中根结点指针 root，还多了指针 first，用以指向中序遍历序列中第一个结点的地址。

建立中序线索树的算法

中序遍历算法基础上改造：

- ▶ 用一个 `pre` 指针，指向当前访问结点的前一结点，最初 `pre=NULL`。
- ▶ 中序遍历时，对当前访问结点 `p`：
 - 如果 `p` 没有左子，令其左子指针指向其直接前驱，即 `p->left = pre`，并置 `p` 的线索标志 `p->leftFlag = 1`；
 - 如果 `pre` 为空，将 `p` 赋给属性 `first`。
 - 如果 `pre` 不为空，且 `pre` 所指结点没有右子，令其右子指针指向其直接后继，即 `pre->right = p`，并置 `pre` 的线索标志 `pre->rightFlag = 1`。
- ▶ 最后访问的结点，其右子必为空，将其改为线索，即 `p->rightFlag = 1`。

线索化标准存储的二叉树类模板的定义

BTree

```
#ifndef BTREE_H_INCLUDED
#define BTREE_H_INCLUDED
#include <iostream>
#include "seqStack.h"
#include "seqQueue.h"
using namespace std;
//BTree类的前向说明
template <class elemType>
class BTree;
template <class elemType>
class Node
{
    friend class BTree<elemType>;
private:
    elemType data;          Node *left, *
                           right;
```

```
int leftFlag; //用于标识是否线索, 0
               时为孩子, 1时为线索
int rightFlag; //用于标识是否线索, 0
               时为孩子, 1时为线索
public:
    Node(){left=NULL; right=NULL;
           leftFlag = 0; rightFlag=0;};
    Node(const elemType &e, Node* L=NULL,
          Node *R=NULL)
    { data = e;
      left=L; right=R; leftFlag = 0;
        rightFlag=0;
    };
};
```


线索化标准存储的二叉树类模板的定义

BTree

```
template <class elemType>
class BTree
{
    private:
        Node<elemType> *root;
        Node<elemType> *first; //线索中的首结点

    public:
        BTree(){root=NULL; first=NULL; }
        void createTree(const elemType &stopFlag); //创建一棵二叉树
        bool isEmpty () { return (root == NULL); } // 二叉树为空返回 true, 否则返回 false
        void ThreadMid(); // 建立中序线索
        void ThreadMidVisit(); //在中序线索树上进行中序遍历
        void ThreadMidPreVisit(); //在中序线索树上进行前序遍历
};
```

建立中序线索树算法实现

ThreadMid

```
template <class elemType>
void BTree<elemType>::ThreadMid()
{
    if (!root) { first=NULL; return;}
    seqStack<Node<elemType> *> s1;
    seqStack<int> s2;
    Node<elemType> *p, *pre;
    int zero=0, one=1;
    pre = NULL;    first = NULL;
    s1.push(root); s2.push(zero);

    while (!s1.isEmpty())
    {
        flag = s2.top(); s2.pop();
        p = s1.top(); //读取栈顶元素
```

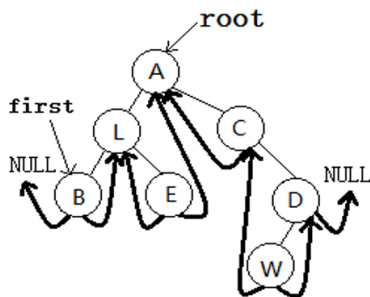
```
        if (flag==one)
        {
            s1.pop();
            if (!first) first = p;
            if (p->right)
                //有右子压右子，没有进入下一轮循环
            {
                s1.push(p->right);    s2.push(
                    zero);    }
            //加中序遍历线索
            if (!p->left)
            {
                p->leftFlag = 1; p->left = pre
                    ; }
            if (pre && (!pre->right))
            {
                pre->rightFlag=1; pre->right =
                    p;    }
            pre = p;
        }
    }
```

建立中序线索树算法实现

ThreadMid

```
else
{   s2.push(one);
    if (p->left)
        //有左子压左子
        {   s1.push(p->left); s2.push(zero); }
    }
} //while
//遍历序列中最后一个结点后继为空
p->rightFlag=1;
}
```

在中序线索树上进行中序遍历算法的思路



中序遍历序列: BLEACWD

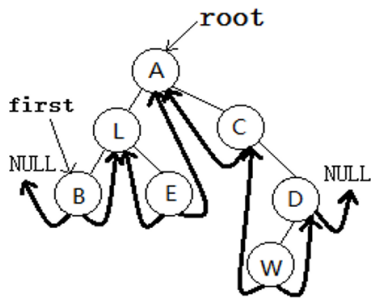
- ① 将当前结点 p 设为 $first$, 即 $p=first$ 。
- ② 如果 p 为空, 结束。
如果 p 不为空, 访问 p 所指结点。
- ③ 如果 p 有右子, 沿其右子一路左分支走下去, 令下一个访问结点 p 指向其最左侧结点。
如果 p 无右子, 令 p 指向右子域中的后继线索, 它即下一个访问结点。
- ④ 转向 2)。

在中序线索树上进行中序遍历算法实现

ThreadMidVisit

```
template <class elemType>
void BTree<elemType>::ThreadMidVisit()
{
    if (!first) return;
    Node<elemType> *p;    p = first;
    while (p)
    {
        cout<<p->data;
        //找p的后继元素
        if (p->rightFlag==0) //如果有右子
        {
            p = p->right;
            //沿右子的左分支一路向左
            while ((p->leftFlag==0))
                p = p->left; //左子为空时必为线索
        }
        else p = p->right; //无右子直接用后继线索
    }
    cout<<endl; }
```

在一棵中序线索树上进行前序遍历算法的思路



中序遍历序列: BLEACWD

- ① 将当前结点 p 置为根 $root$ 。
- ② 如果 p 不为空, 访问 p 所指结点。转向 3) 找下一个访问结点。
- ③ 如果 p 有左子, 下一个访问结点为其左子, 令 p 为其左子。转向 2)。
如果 p 有右子, 下一个访问结点为其右子, 令 p 为其右子。转向 2)。
如果 p 无右子, 顺着后继线索一直往下找, 直到找到右子。
如果右子为空, 遍历结束。如果右子不为空, 右子即下一个访问结点。

在中序线索树上进行前序遍历算法实现

ThreadMidPreVisit

```
template <class elemType>
void BTree<elemType>::ThreadMidPreVisit()
{
    Node<elemType> *p;    p = root;
    while (p)
    {
        cout<<p->data;
        if (p->leftFlag==0)    p = p->left;
        else
        {
            if (p->rightFlag==0)    p = p->right;
            else
            {
                while (p&&(p->rightFlag==1))
                {
                    p = p->right;
                    if (!p) return;
                    p = p->right;
                }
            }
        }
        cout<<endl;    }
}
```

- ▶ 树
- ▶ 二叉树
- ▶ 遍历序列确定二叉树
- ▶ 最优二叉树
- ▶ 树和森林
- ▶ 二叉线索树 *
- ▶ 表达式树 *
- ▶ 等价关系 *
- ▶ 优先级队列

用二叉树表示的表达式就称为表达式树。

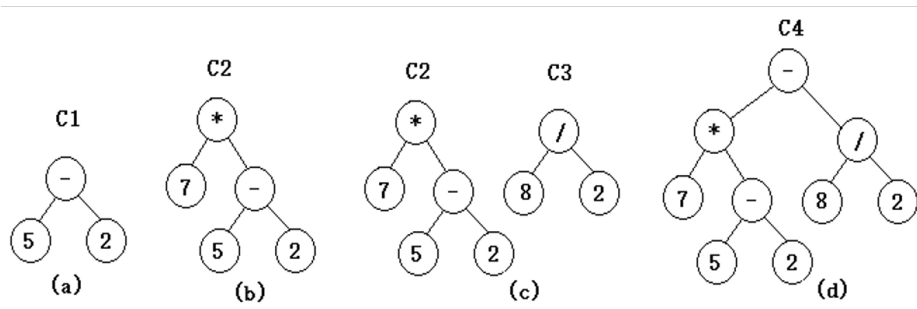
解决三个问题：

- ▶ 如何用二叉树表示表达式
- ▶ 如何根据普通表达式建立表达式树
- ▶ 如何根据表达式树计算表达式的值

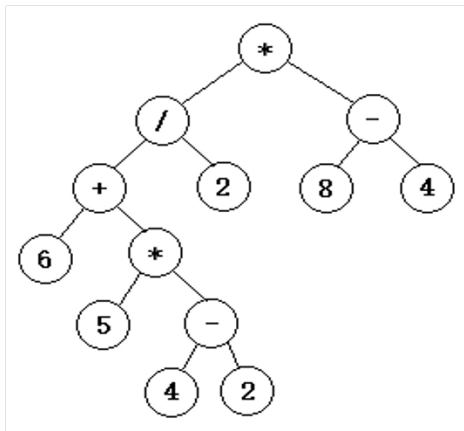
表达式 $7*(5-2)-8/2$ 构建表达式树的过程

按照计算顺序，操作符作为根，操作数 1 作为左子树，操作数 2 作为右子树。

观察表达式树：1) 没有括号 2) 数字都在叶子上 3) 非叶子都是操作符



表达式 $(6+5*(4-2))/2*(8-4)$ 对应的表达式树



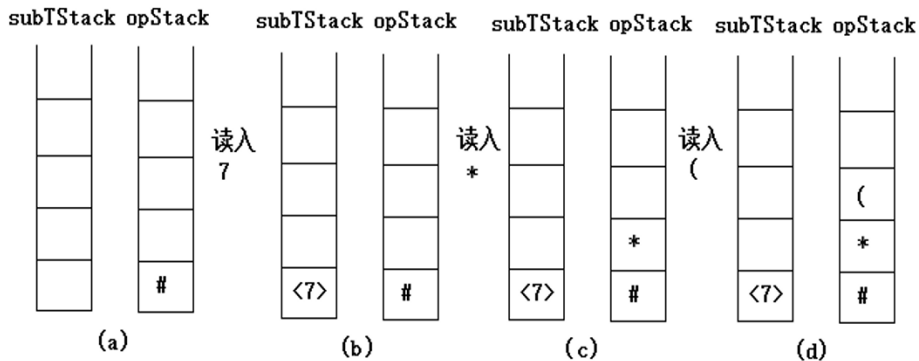
观察手动建立表达式树过程，总结次序规律，发现和产生后缀式类似，表达式树的后序遍历即后缀式。

表达式树的建立

- ▶ 可以借助栈来完成。
- ▶ 为简化：操作符为四则运算，操作数是一位的数字。
- ▶ 结点结构 Node，含有三个字段 data、left、right。
- ▶ 设置两个栈：字符栈（存储操作符）——操作符栈；
指针栈（存储子树的根结点地址）——子树栈。

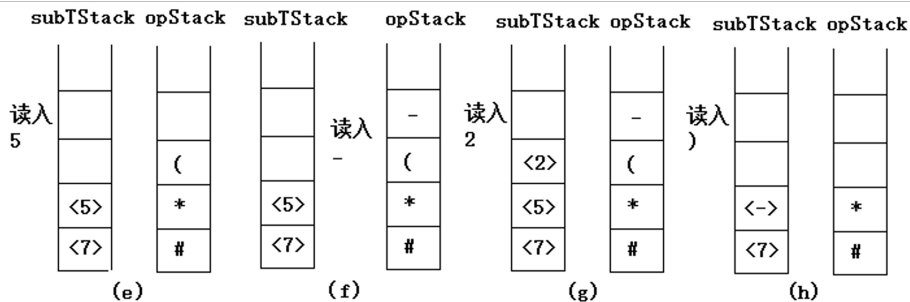
建立表达式 $7*(5-2)-8/2$ 树过程中，两个栈中内容的变化

subTStack 为子树栈，opStack 为操作符栈



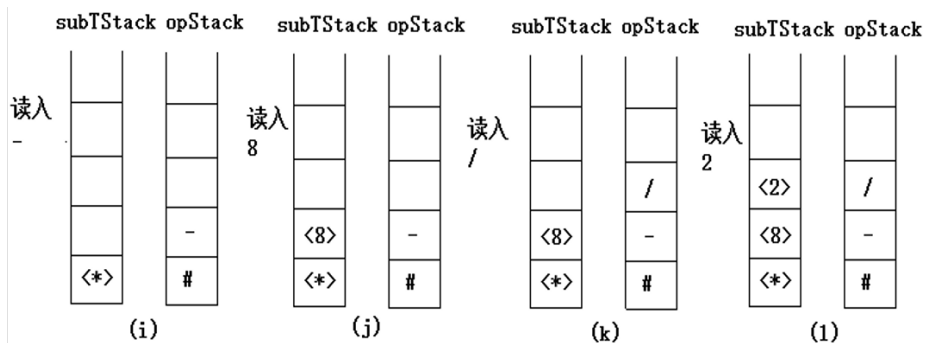
建立表达式 $7*(5-2)-8/2$ 树过程中，两个栈中内容的变化

subTStack 为子树栈，opStack 为操作符栈



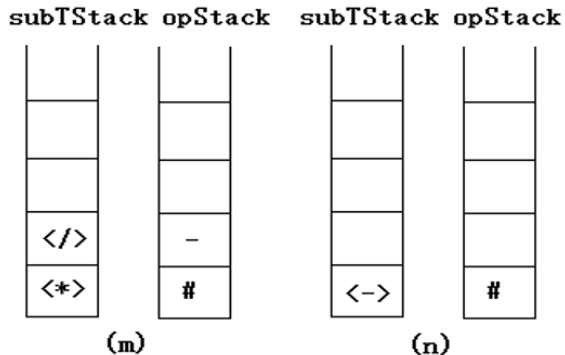
建立表达式 $7*(5-2)-8/2$ 树过程中，两个栈中内容的变化

subTStack 为子树栈，opStack 为操作符栈



建立表达式 $7*(5-2)-8/2$ 树过程中，两个栈中内容的变化

subTStack 为子树栈，opStack 为操作符栈



表达式树的建立算法思路总结：和后缀式的建立逻辑一致

从左向右逐个读取字符串形式的表达式时：

- ① 如果读入的是数字，创建一个结点，把结点当作子树的根，且将其地址压入子树栈；
- ② 如果读入的是左括号，直接将它压入操作符栈；
- ③ 如果读入的是右括号，反复弹操作符栈，直到弹出的是左括号为止；
- ④ 如果读入的是四则运算操作符之一，反复和操作符栈顶元素比优先级，不比栈顶操作符优先级高时，将操作符栈顶元素弹出，直到栈顶元素优先级比读入的操作符低时，将读入的操作符压栈。特别注意：左括号在栈中时，优先级视为最低；左括号即将进站时，优先级视为最高。
- ⑤ 当表达式字符串全部读入后，继续将操作符栈中操作符弹出，直到栈空。最后，子树栈中仅有的一个结点地址就是表达式树的根结点地址。

表达式树的建立算法思路总结

以上操作中，每弹出一个操作符（除左括号），将它作为结点 data，创建一个新的结点，之后从子树栈中弹出两个元素即两个子树的根作为新结点的左右子，最后将新结点地址作为新构成的子树的根地址压入子树栈。

建立表达式树算法的实现

priOver

```
template <class elemType>
int BTree<elemType>::priOver(const char ch1, const char ch2)
//比较操作符优先级的函数，ch1为新读入操作符，ch2为操作符栈栈顶操作符
//当ch1优先级比ch2高，函数返回1；低或相同时返回-1；右、左括号时返回0。
{
    switch (ch1)
    {
        case '(': return 1; //左括号优先级在栈外是最高
        case ')': if (ch2 != '(') return -1;
                    //右括号和栈顶四则运算操作符比，总是优先级最低
                    else return 0; //右括号和栈顶的左括号比，优先级一样

        case '*':
        case '/': if ((ch2 == '*') || (ch2 == '/')) return -1; //相同运算，栈中优先级高
                    else return 1; //只要栈顶不是乘除，优先级肯定比栈顶操作符高

        case '+':
        case '-': //只要栈顶不是hash和左括号，都比栈顶优先级高
                    if ((ch2 == '#') || (ch2 == '(')) return 1;
                    else return -1;
    }
    // 思考：一定要用成员函数吗？
}
```

建立表达式树算法的实现

buildExpTree

//根据一个表达式字符串建立表达式树2

```
template <class elemType>
void BTree<elemType>::buildExpTree(const char
    *exp)
{
    seqStack<char> opStack; //操作符栈
    seqStack<Node<elemType>*> subTStack;
    //子树栈

    Node<elemType> *p, *left, *right;
    char hash = '#', ch;
    opStack.push(hash);
```

```
    while (*exp)
    {
        if ((*exp>= '0')&&(*exp<= '9'))
            //读入数字
            {
                p = new Node<elemType> (*exp);
                subTStack.push(p);
            }
        else //读入操作符 (包括括号)
        {
            ch = opStack.top(); //读opStack栈
                                顶
```

建立表达式树算法的实现

buildExpTree

```
//将opStack栈顶所有比新读入操作符优先级
//高的弹出来处理
while (priOver(*exp,ch)==-1)
{
    opStack.pop(); //opStack栈顶
                  出栈
    right = subTStack.top();
    subTStack.pop();
    left  = subTStack.top();
    subTStack.pop();
    p = new Node<elemType> (ch,
        left, right);
    //构建新操作数结点

    subTStack.push(p); //新操作数
                      压栈
    ch = opStack.top();
}
```

```
//当前opStack栈顶不比新读入的操作符
//优先级高
if (priOver(*exp,ch)==0)
    //优先级一样，即分别为右左括号
    opStack.pop(); //左括号弹出扔掉即可。
else opStack.push(*exp); //
    opStack栈顶
    //操作符比新读入的操作符优先级
    低，
    //新读入操作符直接压栈
};
exp++;
}
```

建立表达式树算法的实现

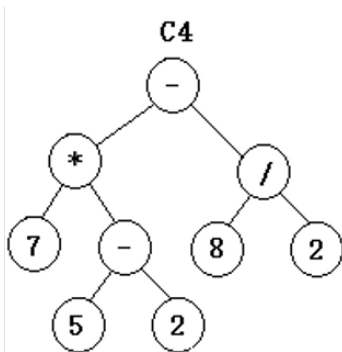
buildExpTree

```
//将opStack栈中所有操作符弹空
ch = opStack.top();
while (ch != '#')
{
    opStack.pop();
    right = subTStack.top(); subTStack.pop();
    left = subTStack.top(); subTStack.pop();
    p = new Node<elemType>(ch, left, right);
    subTStack.push(p);

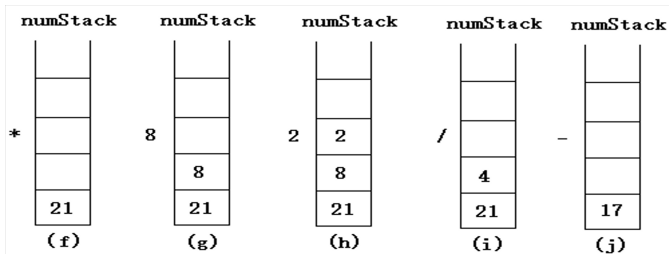
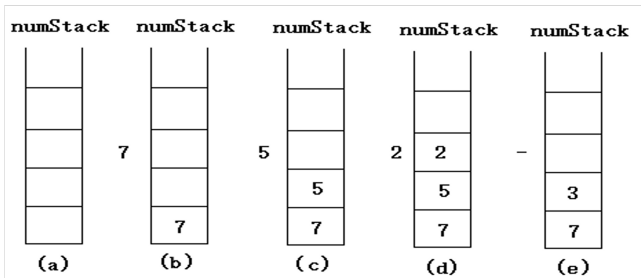
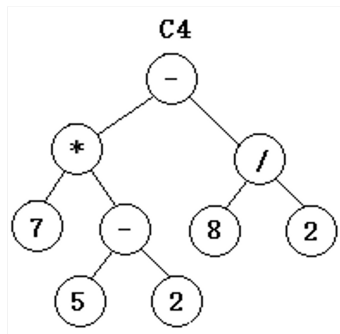
    ch = opStack.top();
}
//操作数栈numStack中剩余的唯一的
//元素即二叉树的根
root = subTStack.top(); subTStack.pop();
}
```

根据表达式树计算表达式的值

- ▶ 可以用一个操作数栈，然后根据后序遍历“左右根”的思路完成。
- ▶ 后序遍历中，当访问到数字时，将它压入操作数栈；当访问到操作符时，两次弹出操作数栈作为操作数进行对应的运算，结果压入操作数栈。
- ▶ 反复如此，直到表达式树后序遍历完毕。



示例



计算表达式树算法的实现

calExpTree

//用后序遍历方法计算一个表达式树的值

```
template <class elemType>
int BTree<elemType>::calExpTree()
{
    //后序遍历
    if (!root) return 0;

    Node<elemType> *p;
    seqStack<Node<elemType> *> s1;
    seqStack<int> s2;

    seqStack<int> numStack;
```

```
int zero=0, one=1, two=2;
int flag, num, num1, num2;

s1.push(root); s2.push(zero);
while (!s1.isEmpty())
{
    flag = s2.top(); s2.pop();
    p = s1.top();
    switch(flag)
    {
```

计算表达式树算法的实现

calExpTree

```
case 2:
    s1.pop();
    //cout << p->data;
    //访问结点时，开始处理：
    //见数字压数字栈，见操作符弹
    //两数字计算，计算结果压数字栈
    if ((p->data>= '0')&&(p->data<= '9'))
    {
        num = p->data- '0';
        numStack.push(num);
    }
```

```
else
{
    num2 = numStack.top(); numStack.pop();
    ;
    num1 = numStack.top(); numStack.pop();
    ;
    switch (p->data)
    {
        case '+': num = num1+num2; break;
        case '-': num = num1-num2; break;
        case '*': num = num1*num2; break;
        case '/': num = num1/num2; break;
    };
    numStack.push(num);
}
break;
```

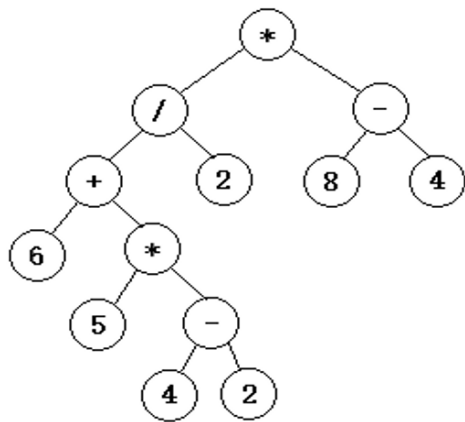
计算表达式树算法的实现

calExpTree

```
case 1: s2.push(two);
        if (p->right)
        {
            s1.push(p->right);
            s2.push(zero);
        }
        break;
case 0: s2.push(one);
        if (p->left)
        {
            s1.push(p->left);
            s2.push(zero);
        }
        break;
```

```
        }; //switch
    } //while
    //get the result of the expression tree
    num = numStack.top(); numStack.pop();
    return num;
}
```

$(6+5*(4-2))/2*(8-4)$ 的表达式树与后缀式

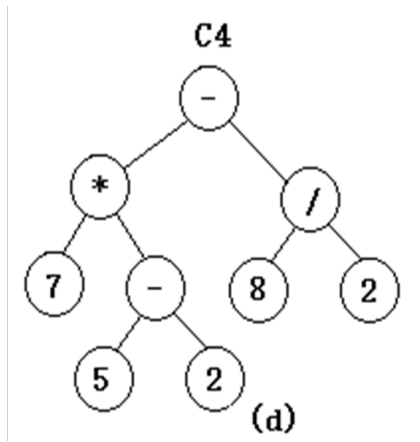


该表达式树的后序遍历为：

6 5 4 2 - * + 2 / 8 4 - *

这正好是该表达式的后缀形式

思考：如何根据以下的表达式树构建表达式？



$$7*(5-2)-8/2$$

- ▶ 树
- ▶ 二叉树
- ▶ 遍历序列确定二叉树
- ▶ 最优二叉树
- ▶ 树和森林
- ▶ 二叉线索树 *
- ▶ 表达式树 *
- ▶ 等价关系 *
- ▶ 优先级队列

等价关系

假设有一个集合 S 上的关系 R $\forall x_1, x_2 \in S$ $x_1 R x_2$ 为真或者假, 且满足以下性质:

自反性: $\forall x_1 \in S, x_1 R x_1$ 为真。

对称性: $\forall x_1, x_2$, 如果 $x_1 R x_2$ 为真, 必有 $x_2 R x_1$ 为真。

传递性: $\forall x_1, x_2, x_3$, 如果 $x_1 R x_2$ 为真且 $x_2 R x_3$ 为真, 则 $x_1 R x_3$ 为真。

称 R 是集合 S 上的等价关系。

等价关系示例

- ① 班级作为一个集合，其中 R 是同性别关系。对于班级中的任何两个同学，同性别关系要么是真要么是假。自反性：李力和自己是同性别的，即结果为真；对称性：李力和王强是同性别的，则王强和李力也是同性别的；传递性：李力和王强是同性别的，王强和刘平是同性别的，则李力和刘平也是同性别的。
- ② 一个装满彩色球的盒子，里面有赤、橙、黄、绿、青、蓝、紫 7 种颜色的小球，盒中任意两个小球之间的同色关系 R 就满足等价关系。
- ③ 生活中常说的等于关系也是一个等价关系，小于关系不是一个等价关系，因为小于关系不满足对称性。

等价类

$\forall x_1 \in S$, 其所属等价类是集合 S 的一个子集 S_1 。

这个子集有这样的特点: $x_1, x_2 \in S_1$, 必有 $x_1 R x_2$ 为真。

例如: 第一个例子中, 男生集合和女生集合各是一个等价类,

第二个例子中, 同种颜色的球构成的子集是一个等价类, 7 中颜色共有 7 个等价类。

不相交集

集合 S 的所有等价类形成了集合 $A = \{s_1, s_2, \dots, s_m\}$, 显然有 $\forall s_i \in A, s_i \neq \emptyset, \forall s_i, s_j \in A, s_i \cap s_j = \emptyset, \cup_{i=1}^m s_i = S$,

因此集合 A 是对集合 S 的一个划分。划分中的每个子集即集合 A 中的各个元素, 它们本身又是一个集合, 称为不相交集。

不相交集的基本操作分为：合并和查找，故不相交集又称为并查集 (union-finds sets)。

合并 $\text{union}(s_i, s_j)$ ：是对两个不相交集进行合并，使之成为一个新的、更大的不相交集。当分属于两个不相交集的元素间添加了等价关系，根据传递性，也要合并这两个不相交集。特别地：当有元素 x 插入时，可以通过把单个元素 x 视作由它自己构成了一个新的不相交集，让新的不相交集和某个已有的不相交集合并，就完成了插入操作。

查找 $\text{find}(x)$ ：是对集合 S 中的元素 x 找到它所属的不相交并，这里即给出不相交并的标志。

存储

不相交集的存储分为：**顺序存储**和**树形存储**。

顺序存储：可以将集合 S 中的所有元素放置在同一个数组中，数组中的每个分量除了存储元素还要存储元素所属的不相交集的标志。

可以看出，下列元素分属于三个不相交集。

data	a	b	c	d	e	f	g	h	k	t		
flag	0	1	2	2	0	1	0	2	2	2		

不相交集的存储分为：顺序存储和树形存储。

树形存储：将集合 S 中的所有元素放置在同一个数组中，而数组中的每个分量除了存储元素，还要存储元素的父结点下标，这种存储方式称双亲表示法，它是一种树形存储。特殊地：当某个元素的父结点下标为-1，这个元素就是树根。

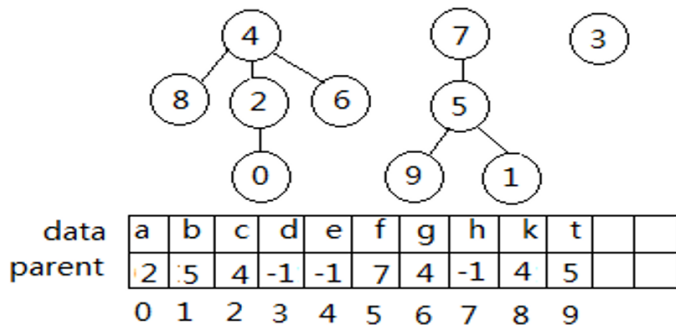
- 每个不相交集用一棵树来表示，然后用树根下标表示所属不相交集的标志。
- 集合中的任何一个元素沿着父结点字段都可以追溯到根，找到所属的不相交集合的标志。
- 代表不同不相交集的树组成一片森林。

思考：树中每个结点为啥只存 parent？

存储

不相交集的存储分为：顺序存储和树形存储。

树形存储：



树形存储的不相交集的基本操作算法

查找：对于一个元素只需要沿着这棵树向上找到树根，得到树根的下标，就完成了查找任务。时间花费是这棵树的高度。显然树的高度越低越好，最低时一个不相交集可以退化为两层，一个元素作为树根，其余元素作为树根的儿子结点。

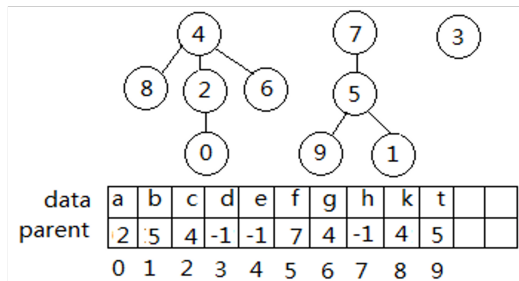
合并：当 a , b 两个不相交集要合并时，可以将 b 中所有元素都当作 a 的根结点的儿子，但时间花费和 b 中元素个数相关。如果简单地把 b 的根作为 a 的根结点的儿子，时间花费是常量阶的。

顺序存储和树形存储的优缺点

- 如果采用顺序存储法，查找、合并都是线性阶 $O(n)$ ；
- 如果采用树形存储方法，查找是树的高度、合并是常量阶 $O(1)$ 。

树形存储法是不相交集的常用物理结构。

data	a	b	c	d	e	f	g	h	k	t		
flag	0	1	2	2	0	1	0	2	2	2		



树形存储的不相交集的基本操作算法的优化

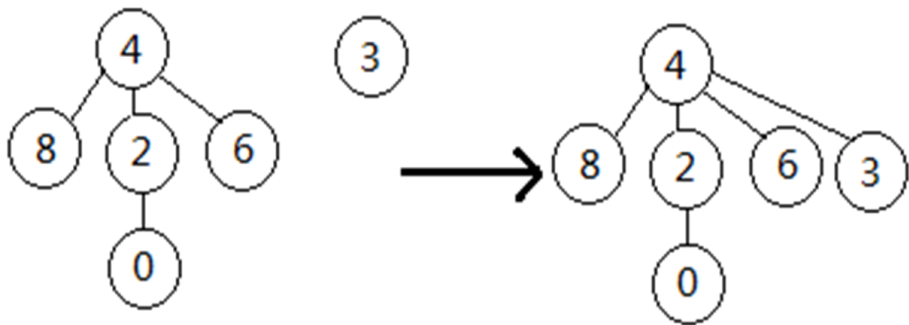
合并操作：按照两个树的高度或者结点规模来判别。

按高度时，将高度小的树并入高度大的树，以高度大的树的根结点作为合并后的树根。这样可以尽量阻止合并后树的高度增加，查找就会因树的高度不增而提高效率。当然，当两个树高度一致时，合并后树高必然增加 1。

按结点规模时，将规模小的树并入规模大的树。这种方法，可使合并后层次号增加 1 的结点个数达到最少。从而达到降低后面平均查找时间的目的。

为方便合并，根结点的父结点不再使用-1，可以用高度或者规模的负数来表示。

合并算法优化后示例

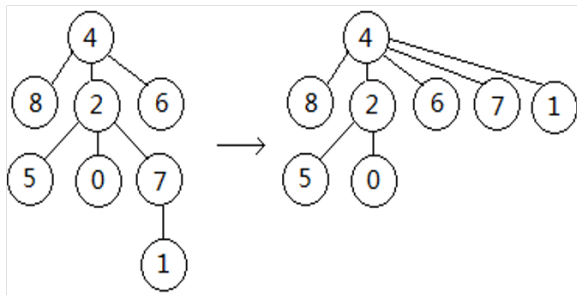


data	a	b	c	d	e	f	g	h	k	t		
parent	2	5	4	4	-3	7	4	-3	4	5		
	0	1	2	3	4	5	6	7	8	9		

树形存储的不相交集的基本操作算法的优化

查找操作：采取越近期访问过的结点越往根结点靠的原则。

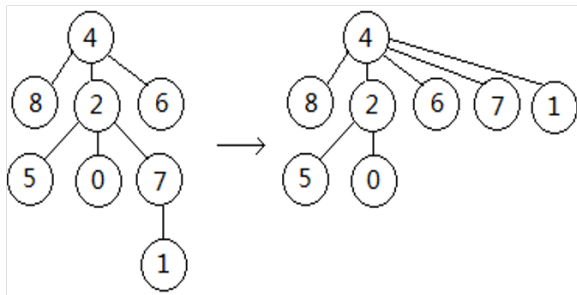
查找时，会沿着查找结点到根一路访问过去。这条查找结点到根结点路径上所有的结点（但不含根结点）全部改为根结点的儿子结点，此方法称为**路径压缩法**。例：查找 1 后



树形存储的不相交集的基本操作算法的优化

路径压缩法的优劣：

查找频率高的结点会集中分布在根部附近，查找高频结点会更快，但如果每个元素具有平均查找概率，反而会因为查找后多出来的移动操作，耗费了多余的时间。



- ▶ 树
- ▶ 二叉树
- ▶ 遍历序列确定二叉树
- ▶ 最优二叉树
- ▶ 树和森林
- ▶ 二叉线索树 *
- ▶ 表达式树 *
- ▶ 等价关系 *
- ▶ 优先级队列

优先级队列

- ▶ 无论是顺序存储还是链式存储，进、出队都有一个操作时间复杂度是 $O(1)$ 而另外一个为 $O(n)$ 。
- ▶ 假设元素的值用其优先级的级别值来标识，级别值小的优先级高，反之则优先级低。
- ▶ 为了提高效率，优先级可以用一种特殊的二叉树：堆来表示，进出队的效率可以统一到 $O(\log_2 n)$ 。

堆的概念：

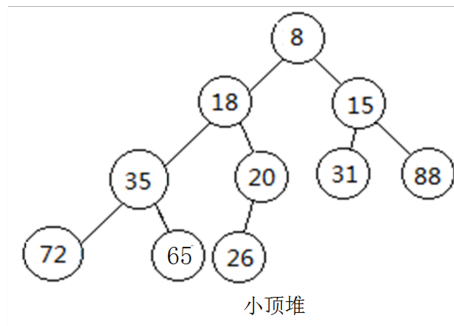
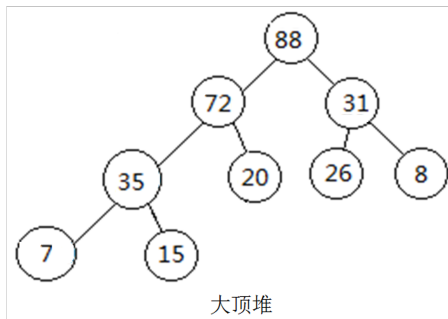
一个完全二叉树中，任意一个结点的值比其左右子结点值都大，称为大顶堆。

一个完全二叉树中，任意一个结点的值比其左右子结点值都小，称为小顶堆。

大顶堆和小顶堆都称为堆。

优先级队列

堆的示例：



堆的特性

二个

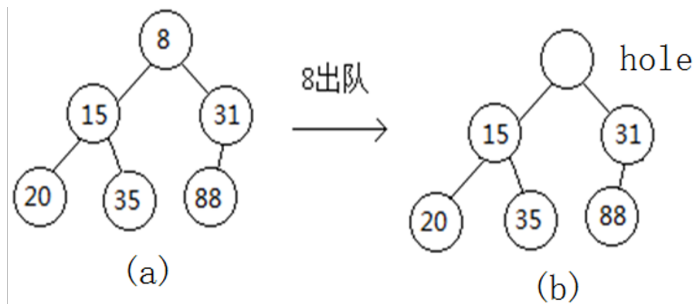
- ① 结构性（完全二叉树）
- ② 有序性（小顶堆：结点值比左右子小。
大顶堆：结点值比左右子大）

假设结点值越小，优先级越大，则可利用小顶堆来表示一个优先级队列。

优先队列的基本操作算法讨论

- ▶ 出队：直接读取堆顶（即二叉树的根），时间花费为 $O(1)$ ；摘取堆顶后，将尾部元素写入堆顶，并对堆顶位置上的元素做调整，使之成为新的小顶堆。
时间花费为此完全二叉树的高度 $O(\log_2 n)$ ，即出队的时间复杂度为 $O(\log_2 n)$ 。
- ▶ 进队：将新元素加入到序列尾部成为最后的叶子结点，向上检查父结点，如果新结点值不小于父结点，结束检查；如果新结点值小于父结点，交换两者的值，并进一步往更高层祖先检查比较，直到不小于当前父结点或无父结点。因此进队的时间花费也是完全二叉树的高度 $O(\log_2 n)$ 。

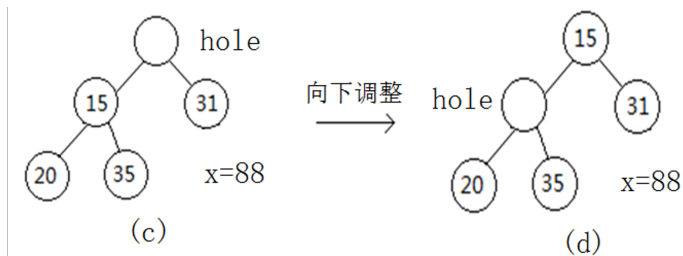
优先队列出队示例



问题：谁去补这个洞？按照堆的结构性要求，将 88 放入即可。

88 放入，可能破坏了有序性，但其左右子树都是小顶堆，可采用将洞反复向下调整，直到洞的左右孩子都比 88 大或者洞为叶子，88 放入洞中，结果满足有序性。

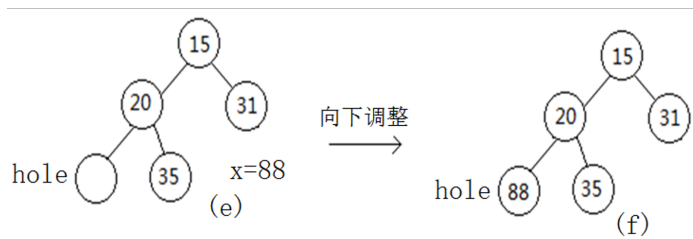
优先队列出队示例



问题：谁去补这个洞？按照堆的结构性要求，将 88 放入即可。

88 放入，可能破坏了有序性，但其左右子树都是小顶堆，可采用将洞反复向下调整，直到洞的左右孩子都比 88 大或者洞为叶子，88 放入洞中，结果满足有序性。

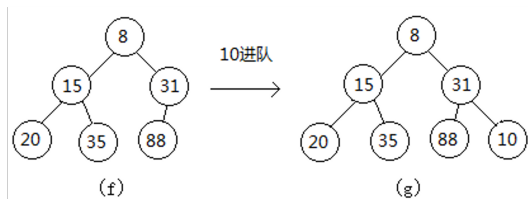
优先队列出队示例



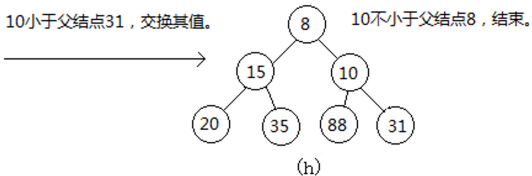
问题：谁去补这个洞？按照堆的结构性要求，将 88 放入即可。

88 放入，可能破坏了有序性，但其左右子树都是小顶堆，可采用将洞反复向下调整，直到洞的左右孩子都比 88 大或者洞为叶子，88 放入洞中，结果满足有序性。

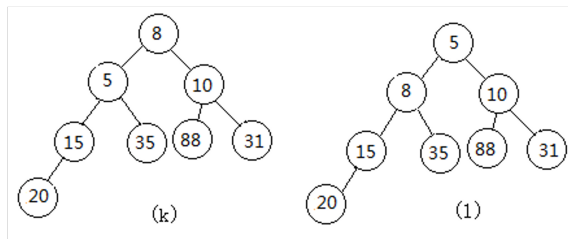
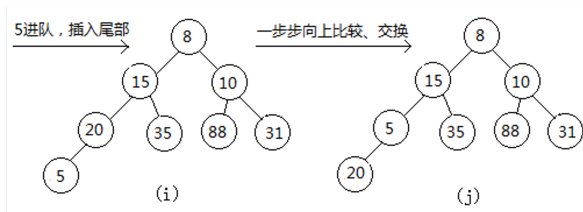
优先队列进队示例 1:



10小于父结点31，交换其值。



优先队列进队示例 2:



建立优先级队列

- ▶ 优先级级别值最小的优先级最高，建立优先级队列即建立小顶堆。
- ▶ 将存于数组中的原始序列看作是一棵完全二叉树的顺序存储。
- ▶ 按照堆的概念调整之，使之成为一个小顶堆。

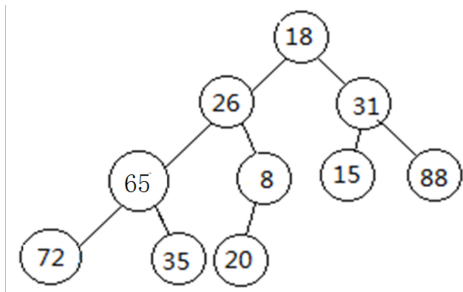
调整策略是：在一个结点的左子树和右子树都是小顶堆的基础上，调整该结点使之成为小顶点。

建立小顶堆

已知数据序列 18、26、31、65、8、15、88、72、35、20

将之看作顺序存储的完全二叉树（结构性）

进行调整，使之满足有序性



建立小顶堆

调整策略：

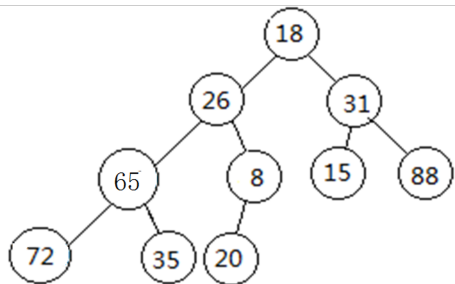
假设一个结点的左子树和右子树都是小顶堆，调整之，使以该结点为根的完全二叉树也是小顶堆。（递归？）

一种调整方法：

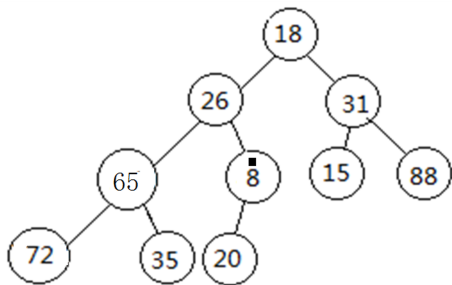
按照以上原则，对序列从后往前逐一做元素检查、调整使得以该元素为根的二叉树满足小顶堆的定义。

叶子结点自然满足小顶堆定义，故从倒数第一个非叶子结点开始逐一调整。

建立小顶堆

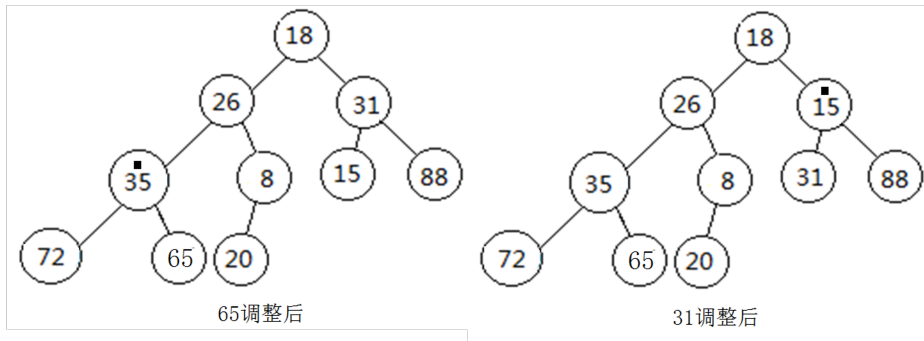


原始序列

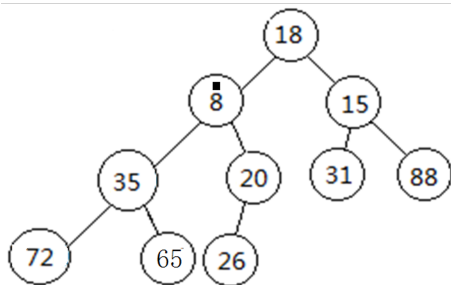


8调整后

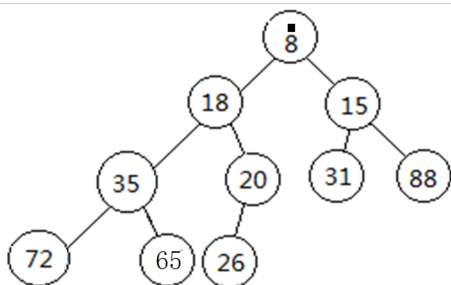
建立小顶堆



建立小顶堆



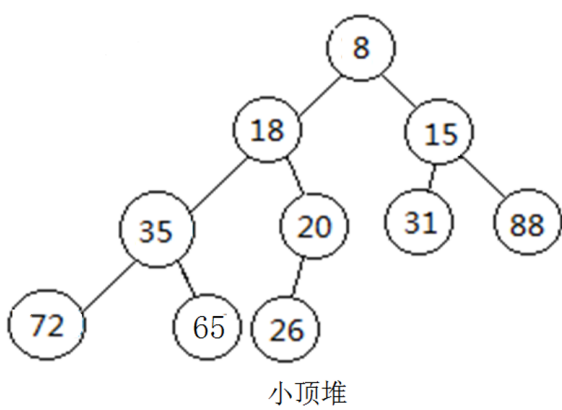
26调整后



18调整后 得小顶堆

建立小顶堆

一个小顶堆，可表示一个优先级队列，堆顶是优先级最高的元素。



建堆时间复杂度分析

从形式上看时间复杂度是 $O(n\log_2 n)$, 但实际可达 $O(n)$ 。

假设堆的高度为 $h+1$, 总的元素个数为 n 。当堆是一个满二叉树时, 有:

$$n = 2^{h+1} - 1$$

观察此堆, 从后往前逐个检查并调整各个非叶子结点时, 比较并调整的最大次数为以该结点为根的堆的高度-1。

建堆时间复杂度分析

第一批非叶子结点在倒数第二层，倒数第二层总的结点个数为 2^{h-1} 个，各结点的比较调整最大次数为 1；

倒数第三层总的结点个数为 2^{h-2} 个，各结点的比较调整最大次数为 2；

根结点这层的结点个数为 $2^{h-h} = 2^0 = 1$ 个，结点比较调整最大次数为 h 。

建堆时间复杂度分析

故总的比较调整次数最多为：

$$\begin{aligned}t &= \sum_{i=h-1}^0 2^i(h-i) \\&= h + 2(h-1) + 4(h-2) + \cdots + 2^{h-2}(2) + 2^{h-1}(1) \\2t &= 2h + 4(h-1) + 8(h-2) + \cdots + 2^{h-1}(2) + 2^h(1) \\&= 0 + 2h + 4(h-1) + 8(h-2) + \cdots + 2^{h-1}(2) + 2^h(1)\end{aligned}$$

建堆时间复杂度分析

$$t = 2t - t$$

$$= -h + 2 + 4 + 8 + \cdots + 2^{h-1} + 2^h$$

$$= -h - 1 + 1 + 2 + 4 + 8 + \cdots + 2^{h-1} + 2^h$$

$$= -(h + 1) + \frac{1 - 2^{h+1}}{1 - 2}$$

$$= -(h + 1) + 2^{h+1} - 1$$

$$= n - (h + 1)$$

又 h 为 n 的对数阶，故建堆的时间复杂度为 $O(n)$ 。

优先级队列类模板的定义

priorityQueue

```
#ifndef PRIORITYQUEUE_H_INCLUDED
#define PRIORITYQUEUE_H_INCLUDED
class illegalSize{};
class outOfBound{};
template <class elemType>
class priorityQueue
{    private:
    elemType *array;    int
        maxSize, currentLen;
    void doubleSpace(); //扩展队
        列元素的存储空间
    void adjust(int hole); //调
        整下标为hole的元素, 使
        成为小顶堆
    void build(int r); //递归调
        整
```

```
public:
    priorityQueue(int size=10); //建立空优先级队列
    priorityQueue(elemType a[], int n); //建立非空
        优先级队列, 即建小顶堆
    bool isEmpty() {return currentLen==0;}; //判
        断队空否
    bool isFull() {return (currentLen==maxSize-1)
        }; //判断队满否
    elemType front(); //读取队首元素的值, 队首不变
    void enqueue(const elemType &x); //将x进队, 成
        为新的队尾
    void dequeue(); //将队首元素出队
    ~priorityQueue(){delete []array; }; //释放队
        列元素所占据的动态数组
};
```

优先级队列类模板成员函数的实现

priorityQueue

```
template <class elemType>
priorityQueue<elemType>::priorityQueue (int
    size) //建立空优先级队列
{
    array = new elemType[size]; //申请实际的
        队列存储空间
    if (!array) throw illegalSize();
    maxSize = size;
    currentLen = 0;
}
```

```
template <class elemType>
priorityQueue<elemType>::priorityQueue (
    elemType a[], int n) //建立优先级队列
{
    if (n<1) throw illegalSize();
    array = new elemType[n+10]; //申请实际的
        队列存储空间, 多10, 支持入队
    if (!array) throw illegalSize();
    maxSize = n+10;      currentLen = n;
    for (i=1; i<=n; i++)    array[i] = a[i
        -1];
    for (i=n/2; i>=1; i--)    adjust(i);
        //首次建立小顶堆
}
```

优先级队列类模板成员函数的实现

priorityQueue

```
template <class elemType>
priorityQueue<elemType>::priorityQueue (
    elemType a[], int n) //另一种方法建立优先级队列
{
    if (n<1) throw illegalSize();
    array = new elemType[n+10]; //申请实际的队列存储空间, 多10, 支持入队
    if (!array) throw illegalSize();
    maxSize = n+10;      currentLen = n;
    for (int i=0; i<n; i++) array[i+1]=a[i];
    build(1); //建立小顶堆
}
```

```
template <class elemType>
void priorityQueue<elemType>::build (int r)
    //调整为优先级队列
{
    if (r>currentLen/2) return;
    build(2*r); //将左子树调整为堆
    build(2*r+1); //将右子树调整为堆
    adjust(r);
}
```

优先级队列类模板成员函数的实现

priorityQueue

```
template <class elemType>
void priorityQueue<elemType>:: adjust(int
    hole)
//反复向下调整 hole 的位置
{
    int minChild;      elemType x;
    x = array[hole];
    while (true)
    {
        minChild = 2*hole;  //hole的左子下标
```

```
        if (minChild>currentLen) break;
        if (minChild+1<=currentLen) //hole还有右子
        if (array[minChild+1]< array[minChild])
            minChild++; //右子最小

        if (x<array[minChild]) break;
        array[hole] = array[minChild];
        hole = minChild; //继续向下调整
    }

    array[hole] = x;

    //比较次数最多为：二叉树的高度
```

优先级队列类模板成员函数的实现

priorityQueue

```
template <class elemType>
elemType priorityQueue<elemType>::front() //
    读取队首元素的值，队首不变
{
    if (isEmpty()) throw outOfBound();
    return array[1]; }


```

```
template <class elemType>
void priorityQueue<elemType>::deQueue() //将
    队首元素出队
{
    if (isEmpty()) throw outOfBound();
    array[1] = array[currentLen];
    currentLen--;
    adjust(1);
}


```

```
template <class elemType>
void priorityQueue<elemType>::enqueue(const
    elemType &x) //将x进队
{
    if (isFull()) doubleSpace();
    int hole ; //hole位置向上调整
    currentLen++;
    for (hole= currentLen; hole > 1 && x <
        array[ hole/ 2 ]; hole /= 2 )
        array[ hole ] = array[ hole / 2 ];
    array[ hole ] = x;
}

//比较次数最多为二叉树高度，故入队时间效率为
 $O(\log_2 n)$ 


```

单服务台的排队系统

- 在单服务台系统中，先到达的顾客先获得服务，也肯定先离开。到达和离开的次序是一致的。
- 只需要一个普通队列保存顾客到达信息。

多服务台的排队系统

- 在多服务台系统中，先到达的顾客先获得服务，但后获得服务的顾客可能先离开；
- 事件处理的次序可能是：顾客 1 到达、顾客 2 到达、顾客 2 离开、顾客 3 到达、顾客 1 离开、顾客 3 离开……、顾客 n 到达、顾客 n 离开、……；
- 发生时间早的事件先处理，发生时间晚的事件后处理。因而需要一个**优先级队列**存放事件，**事件的优先级就是发生的时间**。

多服务台的排队系统

- 所以一共需要两个队列：

1. 普通队列：保存顾客到达时间，当前队列中的元素由还未被窗口服务的等待顾客构成。
2. 优先队列：保存事件队列，它由所有正在各个窗口被服务的顾客构成。这些顾客有到达和离开两种。

- 模拟开始时，产生所有的到达事件，存入优先级队列。
- 模拟器开始处理事件：
 - ◇ 从队列中取出一个事件。这是第一个顾客的到达事件。生成所需的服务时间。当前时间加上这个服务时间就是这个顾客的离开时间。生成一个在这个时候离开的事件，插入到事件队列。
 - ◇ 同样的模拟器从队列中取出的事件也可能是离开事件，这时只要将这个离开事件从队列中删去，那么为他服务的服务台就变成了空闲状态，可以继续为别的顾客服务。

多服务台的排队系统模拟过程

1. 产生 CustomNum 个顾客的到达事件;
2. 按时间的大小存入事件队列;
3. 置等待队列为空;
4. 置所有柜台为空闲;
5. 设置等待时间为 0。

多服务台的排队系统模拟伪代码

多服务台的排队系统

```
while (事件队列非空) {  
    队头元素出队；设置当前时间为该事件发生的时间；  
    switch(事件类型) {  
        case 到达: if (柜台有空) {  
            柜台数减1；生成所需的服务时间；  
            修改事件类型为“离开”；  
            设置事件发生时间为当前时间加上服务时间；  
            重新存入事件队列；}  
        else 将该事件存入等待队列；  
        case 离开: if (等待队列非空) {  
            队头元素出队；统计该顾客的等待时间；  
            生成所需的服务时间；修改事件类型为“离开”；  
            设置事件发生时间为当前时间加上服务时间；  
            存入事件队列；}  
        else 空闲柜台数加1；  
    }  
}  
计算平均等待时间；返回；
```

- 设计一个模拟类，告诉它顾客到达时间间隔的分布、服务时间的分布、模拟的柜台数以及想要模拟的顾客数。该模拟类能提供在本次服务中顾客的平均等待时间是多少。
- 这个类应该有两个公有函数：构造函数接受用户输入的参数，另外一个就是执行模拟并最终给出平均等待时间的函数`avgWaitTime`。这个类要保存的数据就是模拟的参数。

模拟类的定义

Simulator

```
class simulator
{
    int noOfServer;    //服务台个数
    int arrivalLow;    //到达间隔时间的下界
    int arrivalHigh;   //到达间隔时间的上界
    int serviceTimeLow; //服务间隔时间的下界
    int serviceTimeHigh; //服务间隔时间的上界
    int customNum;     //模拟的顾客数
    struct eventT
    {
        int time;    //事件发生时间
        int type;    //事件类型。0为到达，1为离开
        bool operator< (const eventT &e) const {return time < e.time;}
    };
public:
    simulator();
    int avgWaitTime();
};
```

构造函数的实现

构造函数

```
simulator:: simulator()  
{  
    cout << "请输入柜台数：" ; cin >> noOfServer;  
    cout << "请输入到达时间间隔的上下界  
        (最小间隔时间 最大间隔时间)：" ;  
    cin >> arrivalLow >> arrivalHigh;  
    cout << "请输入服务时间的上下界  
        (服务时间下界 服务时间上界)：" ;  
    cin >> serviceTimeLow >> serviceTimeHigh;  
    cout << "请输入模拟的顾客数：" ;  
    cin >> customNum;  
    srand(time(NULL));  
}
```


avgWaitTime() 的实现

AvgWaitTime

```
int simulator:: avgWaitTime()
{
    int serverBusy = 0; //正在工作的服务台数
    int currentTime; //记录模拟过程中的时间
    int totalWaitTime = 0; //模拟过程中所有
        顾客的等待时间的总和
    linkQueue<eventT> waitQueue; //顾客等待
        队列
    priorityQueue<eventT> eventQueue; //事件
        队列
    eventT currentEvent;
    int i;
    currentEvent.time = 0;
    currentEvent.type = 0;
    for (i = 0; i < customNum; ++i) //生成初
        始的事件队列
    {
        currentEvent.time += arrivalLow
```

```
        +(arrivalHigh - arrivalLow + 1)
        *rand() / (RAND_MAX + 1);
        eventQueue.enqueue(currentEvent);
    }
    //模拟过程
    while (!eventQueue.isEmpty())
    {
        currentEvent = eventQueue.dequeue();
        currentTime = currentEvent.time;
        switch (currentEvent.type)
        {
            case 0: //处理到达事件
                break;
            case 1: //处理离开事件
        } //switch结束
    } //while结束
    return totalWaitTime / customNum;
}
```

处理到达事件代码

处理到达事件

```
if (serverBusy != noOfServer)
{
    ++serverBusy;
    currentEvent.time += serviceTimeLow
        + (serviceTimeHigh - serviceTimeLow + 1)
        * rand() / (RAND_MAX + 1);
    currentEvent.type = 1;
    eventQueue.enqueue(currentEvent);
}
else waitQueue.enqueue(currentEvent);
```

处理离开事件代码

处理离开事件

```
if (!waitQueue.isEmpty())
{
    currentEvent = waitQueue.dequeue();
    totalWaitTime += currentTime - currentEvent.time;
    currentEvent.time = currentTime + serviceTimeLow
        + (serviceTimeHigh - serviceTimeLow + 1)
        * rand() / (RAND_MAX + 1);
    currentEvent.type = 1;
    eventQueue.enqueue(currentEvent);
}
else --serverBusy;
```

用例

```
int main()
{
    simulator sim;
    cout << "平均等待时间: "
         << sim.avgWaitTime() << endl;
    return 0;
}
```

某次执行结果

请输入柜台数：4

请输入到达时间间隔的上下界（最小间隔时间 最大间隔时间）：0 2

请输入服务时间的上下界（服务时间下界 服务时间上界）：2 7

请输入模拟的顾客数：1000

平均等待时间：61

- ▶ 树是讨论的第一种非线性结构。树中要求元素个数大于零，元素之间呈现出上下层之间一对多的层次关系。鉴于树物理存储上的一系列难题，转而讨论另外一种更简单的非线性结构：二叉树。
- ▶ 二叉树中元素个数大于等于零，每个结点最多有两个孩子，且每个孩子都有明确的左右子之分。从元素的个数限制上也可以看出，当元素个数为零时，仍可看作是一棵二叉树，但它不是树。另外二叉树中某个结点即便只有一个儿子，也一定要明确它是左儿子还是右儿子，而不是像有序树一样说它是第一个孩子。综上两个原因，不能简单地说二叉树就是一棵有序树，它们是二种不同的数据结构。

- ▶ 本章详细介绍了二叉树的概念、性质，提出了两种特别的二叉树：满二叉树和完全二叉树。从满二叉树和完全二叉树上，可以看到一些有趣的性质和一些特殊的处理手段。
- ▶ 在讨论二叉树的物理结构时，提出了最适合完全二叉树的顺序存储法以及适合普通二叉树的二叉链表存储法（标准形式）。
- ▶ 在基本操作的实现上，详细讨论了标准形式存储的二叉树的递归和非递归算法。鉴于二叉树结构的递归定义，递归算法对于二叉树中的某些基本操作而言，逻辑上最直观、简单、不容易出错。而非递归算法相对于递归算法而言，是把堆栈的使用从幕后推到了台前，避免了次数众多的递归函数调用，降低了由于函数调用产生的额外时间和空间的开销，提高了算法运行效率。

- ▶ 在众多二叉树的基本操作中，本章将遍历作为重点详细进行了算法设计讨论。事实上可以看出，遍历之外的绝大多数操作都可以在遍历算法的基础上实现。如这棵二叉树中有多少度为 2 的结点、某个结点在二叉树中的第几层、二叉树的高度是多少，等等。
- ▶ 事实上，在现实生活中，能直观对应到二叉树结构的数据是极少的。它更多地可看作是一种和现实生活中无物对应、虚构出来的数据结构。但以它为工具，却可以解决很多实际数据的存储和处理问题。如本章中树和森林的孩子兄弟表示法、哈夫曼编码、优先级队列、表达式树、不相交集等，以及后续章节的二叉查找树、堆排序等都可以利用二叉树来解决。

- 利用小顶堆实现优先级队列，可使其进队、出队操作的时间复杂度达到：

A. $O(1)$ 和 $O(n)$

B. $O(n)$ 和 $O(\log 2n)$

C. $O(\log 2n)$ 和 $O(\log 2n)$

D. $O(\log 2n)$ 和 $O(n)$

- 利用小顶堆实现优先级队列，可使其进队、出队操作的时间复杂度达到:

C

A. $O(1)$ 和 $O(n)$

B. $O(n)$ 和 $O(\log 2n)$

C. $O(\log 2n)$ 和 $O(\log 2n)$

D. $O(\log 2n)$ 和 $O(n)$

- 给出的数据初值为 40, 20, 24, 15, 30, 25, 66, 35, 45, 14, 41, 请给出利用 buildHeap 构造出的最小化堆。

经典考题

- 给出的数据初值为 40, 20, 24, 15, 30, 25, 66, 35, 45, 14, 41, 请给出利用 buildHeap 构造出的最小化堆。

